

Master

July 31, 2026

Forgetting by Design

Trade-offs in LLM Unlearning

Dogukan Baysal

of Girne, Cyprus (24-745-069)

supervised by

Prof. Dr. Harald C. Gall

Melih Catal



University of
Zurich^{UZH}



Master

Forgetting by Design

Trade-offs in LLM Unlearning

Dogukan Baysal



University of
Zurich ^{UZH}



Master

Author: Dogukan Baysal, dogukan.baysal@uzh.ch

Project period: 27.03.2026 - 27.09.2026

Software Evolution & Architecture Lab

Department of Informatics, University of Zurich

Acknowledgements

Abstract

Large Language Models (LLMs) trained on source code may memorize and reproduce sensitive or copyrighted content present in their training data. Machine Unlearning (MU) aims to remove the influence of such data without retraining the model from scratch, but its effectiveness for code generation tasks remains underexplored. This thesis investigates how unlearning outcomes are shaped by the choice of MU technique, the type of content targeted for removal, the size of retain set, and the ordering of forget and retain set objectives. We evaluate Gradient Ascent (GA), Negative Preference Optimization (NPO), and PROD, individually and combined with Gradient Descent (GD) and Kullback Leibler Divergence Minimization (KL), on one general-purpose and one code-specialized LLM. Using a synthetic dataset of functions and classes designed to minimize pretraining overlap, we contrast the unlearning of isolated secrets with the unlearning of entire code units.

Our results show that the favorable forgetting–utility trade-offs observed for secrets do not transfer to complete functions and classes: PROD eliminates utility entirely, GA causes severe degradation once forgetting becomes effective, and NPO preserves utility only by producing limited forgetting. For secrets, in contrast, forgetting can be achieved while largely preserving utility at appropriately chosen configurations, though the best technique differs between models. NPO with KL is most stable across models and epochs and best preserves the correctness of code surrounding the removed secret, while plain forget objectives reach stronger model-specific trade-offs at lower energy cost. We further find that objective ordering influences the trade-off more consistently than retain-set size, with forget-first ordering generally most favorable. These findings show that unlearning behavior depends strongly on the structure of the targeted content, and that current MU techniques are not yet suitable for reliably removing complete code units.

Contents

1	Introduction	1
2	Related Work	3
2.1	Machine Unlearning	3
2.2	Machine Unlearning for Code Generation	3
2.3	Evaluation of Code Unlearning	4
2.4	Dataset Curation for Machine Unlearning	4
3	Methodology	7
3.1	Research Questions	7
3.2	Study Design	8
3.2.1	Dataset	8
3.2.2	Model Selection	12
3.2.3	Unlearning Objectives	13
3.2.4	Evaluation	14
4	Experimental Design	17
4.1	Fine-Tuning	17
4.2	Unlearning	17
5	Results	23
5.1	RQ1: Machine Unlearning Technique Trade-offs	23
5.2	RQ2: Transferability of Machine Unlearning to Code Units	29
5.3	RQ3: Effects of Retain-Set Size and Forget-Retain Objective Ordering	34
6	Limitations and Threats to Validity	37
7	Conclusion and Future Work	39
	Appendix	47

List of Figures

3.1	Overview of the experimental pipeline. Synthetic code is generated and injected with secrets, then split into forget and retain sets. A base model is fine-tuned on this data, unlearned with the forget/retain sets, and evaluated.	8
3.2	Overview of the synthetic dataset construction pipeline. GPT-5.5 generates candidate functions and classes, secrets are then planted into the forget set. All candidates pass through a similarity evaluation, elements that violate the configured similarity thresholds are regenerated.	9
3.3	Pairwise lexical and structural similarity over all 179,700 pairs of the final dataset (log-scaled counts). Left: direction-averaged CodeBLEU; Right: token Jaccard similarity. Both lie entirely below the 0.6 threshold (dashed red), with maxima of 0.555 and 0.534 respectively	13
3.4	Pairwise cosine similarity over all 179,700 pairs for two code embedding models (left: <code>SFR-Embedding-Code-400M_R</code> ; right: <code>Qodo-Embed-1-1.5B</code>). Raw cosine (grey) is compressed into a narrow band near 0.6, illustrating embedding anisotropy. After mean-centering (blue), the distribution re-centers near 0.	14
4.1	Mean generated–target similarity across the base and five training epochs for both models. Secret memorization is measured with chrF and code-unit memorization with BLEU, averaged over 150 examples; higher values indicate greater memorization.	18
4.2	Fine-tuned models checkpoint four performance before unlearning. The left panel shows the percentage of items with at least one exact match among ten generations, while the right reports ForgetEval and UtilityEval pass@10 scores.	19
4.3	Trade-off between forgetting quality and model utility across learning rates for GA, NPO, and PROD. The y-axis shows the reduction in average similarity score relative to the baseline model. The x-axis shows UtilityEval retention relative to the baseline. Each line corresponds to a learning rate, with points representing different epochs. The shaded top-right region indicates the preferred optimal area, where models achieve high forgetting effect while retaining utility.	20
5.1	Utility–forgetting trade-off for the selected secret-setting checkpoints at pass@10. For each model–technique configuration, we consider epochs satisfying the minimum thresholds of 0.80 for both UtilityEval and forget quality and select the epoch maximizing their equally weighted mean. Dark outlined markers and dashed lines identify the model-specific Pareto frontiers.	27
5.2	Measured GPU energy for secret-setting unlearning runs. Bars show the mean across Llama-3.2-3B and Qwen2.5-Coder-3B, error bars span the two model-specific values.	28
5.3	ForgetEval pass@10 failure rates by secret location for the selected secret-setting checkpoints. Bars report the mean failure rate across Llama-3.2-3B and Qwen2.5-Coder-3B, while error bars span the two model-specific rates. Results use the 125 tasks retained after baseline filtering, comprising 57 secrets located in code and 68 in documentation. For each model–technique pair, the checkpoint maximizing the equally weighted mean of UtilityEval and forget quality was selected.	30
5.4	Pass@10 utility–forgetting trade-offs for all secret and code-unit checkpoints across both models. Colors indicate the task, shapes the forget objective, fills the retain objective, and marker sizes the epoch. forget quality is $1 - \text{chrF}$ for secret and $1 - \text{BLEU}$ for code unit; higher values on both axes are preferred.	32

5.5	Effects of forget-retain objective ordering and double retain set size relative to random objective ordering for the secret setting. Performance differences are calculated against the corresponding random-order checkpoint for the same model, objective family, and epoch. Small points represent the six matched observations (two models across three epochs), large markers indicate their means, and horizontal lines show the observed ranges.	35
1	Learning-rate sensitivity of full-parameter GA on Qwen2.5-Coder-3B under the secret unlearning setting. Connected markers represent epochs 1–3.	48
2	Pass@10 utility–forgetting trade-offs for all checkpoints across both models, using 1–normalized character-level Levenshtein similarity as a common forget quality measure.	49

List of Tables

3.1	Overview of the dataset composition. Each component is divided equally between functions and classes.	9
3.2	Composition of the forget set by code-unit complexity and injected-secret type. . .	10
4.1	Unlearning configuration. The combined objective is $\mathcal{L} = \mathcal{L}_{\text{forget}} + \lambda_{\text{ret}} \mathcal{L}_{\text{retain}}$. Values marked $\dagger$ are confirmed against the respective source implementation.	20
4.2	Evaluated unlearning techniques.	21
4.3	Experimental configurations for analyzing retain set size and forget-retain objective ordering for MU.	21
5.1	Secret unlearning results at every epoch. Each result cell reports Forget Quality / UtilityEval, where forget quality uses $1 - \text{chrF}$	25
5.2	ForgetEval pass@10 scores at the best epoch of each technique (in parentheses), chosen per model by the equal-weight mean of Forget Quality pass@10 ($1 - \text{chrF}$) and UtilityEval pass@10, averaged over Llama-3.2-3B and Qwen2.5-Coder-3B. The error denotes the deviation of each model from the mean.	29
5.3	Full-parameter GA relative to LoRA-based GA on Qwen2.5-Coder-3B.	29
5.4	Code-unit unlearning results at every epoch. Each result cell reports Forget Quality / UtilityEval, where forget quality uses $1 - \text{BLEU}$	33
1	Sensitivity of the Secret pass@10 best-epoch selection to a common UtilityEval and Forget Quality threshold τ . Epoch agreement is calculated among configurations remaining eligible at τ , relative to the 0.80/0.80 selection. Frontier retention is relative to its four Pareto-optimal points.	47
2	One-factor-at-a-time sensitivity of the Secret pass@10 selection. One threshold is varied while the other is fixed at 0.80. Epoch agreement is relative to the selected epochs at 0.80/0.80 and is reported among eligible configurations. Frontier retention is relative to the four Pareto-optimal points at 0.80/0.80.	47
3	Sensitivity of the Secret pass@10 best-epoch selection to baseline-derived filtering. Epoch agreement is relative to the filtered selection. Frontier retention is relative to its four Pareto-optimal points.	48

List of Listings

3.1	Synthetic forget function with injected email.	10
3.2	Synthetic forget class with injected API key.	11
5.1	PROD-unlearned generation from Qwen2.5-Coder-3B at epoch 1 (UtilityEval, code-unit setting; highlighted segment). The model generates a file-separator token, hallucinates an unrelated file path, resulting in an unexecutable solution.	31

Introduction

Recently, Large Language Models (LLMs) have become particularly prominent in software engineering tasks such as code generation, bug fixing, and automated testing. Their effectiveness in these tasks largely depends on large-scale raw code corpora, much of which originates from human-generated artifacts [1], including publicly available software repositories, technical forums, and documentation [2, 3]. While such data helps align models with real-world development practices and provides natural examples, it also raises concerns around data security and privacy [4, 5], notably through the presence of personally identifiable information (PII) and malicious code [6, 7]. Even though rigorous filtering is applied before pre-training, existing work shows that these models still memorize and reproduce undesirable content present in the training data [8, 9].

To address this, Machine Unlearning (MU) has been introduced as a mechanism that enables models to forget specific data points or subsets of data upon request [10, 11]. The most straightforward solution to unlearning is to retrain the entire model from scratch while excluding the targeted data [10]. Although this approach provides strong privacy and security guarantees, repeating the full training process for every newly identified data instance is computationally prohibitive [12]. This is a serious limitation because deletion requests may arrive sequentially after model deployment [13] and, under regulations such as the GDPR, must be addressed without undue delay [14]. Unlike retraining from scratch, approximate MU techniques aim to selectively remove the influence of targeted data from a trained model at lower cost [11]. This capability is particularly relevant for compliance with privacy regulations such as the *right to be forgotten*, which entitles individuals to request the erasure of their personal data [15, 16] since it makes the process more practical. However, while MU offers a viable alternative to full retraining, it introduces its own difficulty: existing work on LLMs reports noticeable utility degradation, creating a trade-off between effective forgetting and preserved model utility [11, 17]. Although this trade-off has been studied primarily in general-purpose tasks, its behavior in code generation remains comparatively underexplored, with only a handful of works addressing it directly [18–20]. Addressing this gap requires moving beyond method-level comparisons and examining the underlying factors that shape unlearning behavior. This work considers two such factors. The first is the design of the unlearning process, including the order in which forget and retain data are presented and how the corresponding sets are constructed. The second is the nature of the content being forgotten. A forget set may target isolated string-level artifacts embedded in code, such as hardcoded secrets, or it may target semantically coherent program units such as complete functions and classes. These targets differ in how they are structured and unlearned, yet existing work rarely distinguishes between them. Examining these data-centric design decisions offers a novel perspective on how to achieve effective forgetting for code while maintaining model utility.

In light of these challenges and the limited research on MU for LLMs in code generation tasks, this thesis shifts the focus from evaluating unlearning algorithms in isolation to understanding

how unlearning outcomes are shaped by the interaction between method choice, data selection, and the structure of code content. Specifically, we (1) analyze the impact of various MU techniques, including GA, NPO and PROD on model utility, unlearning effectiveness, and resource consumption trade-offs; (2) compare how these trade-offs are affected by the execution order and composition of the forget and retain sets; and (3) examine how unlearning behavior differs across the type of content targeted for removal, contrasting isolated strings embedded in code (e.g., hardcoded secrets) with semantically coherent program units such as functions and classes.

Related Work

This section reviews prior work relevant to MU and introduces key concepts within the field.

2.1 Machine Unlearning

MU techniques aim to provide a superior trade-off between unlearning efficiency and effectiveness compared to retraining the model from scratch [10]. These methods are roughly categorized into two families: MU with parameter optimization and in-context unlearning [21]. In-context unlearning methods try to achieve the forgetting effect during inference without altering model parameters. This allows black-box unlearning, which can be used even on closed-source models. These techniques are inherently resource-efficient since they do not require any additional training step. In contrast, parameter optimization-based methods achieve unlearning by updating model weights [21, 22] and try to remove the knowledge from the model rather than masking it. Models are fine-tuned using carefully constructed counterexamples, replacement samples, or corrective updates targeting those specific instances. A key idea is to use the influence of unlearning points and minimize the impact these samples have on the model. This shifts the unlearning focus from specific data instances to the features and labels through which data influences parameter updates. For example, Warnecke et al. [23] adopt this perspective by translating data effects into closed-form parameter updates. In this way, the undesired knowledge can be adjusted at the level of concepts or associations, rather than being restricted to individual samples. This allows for the precise removal of the undesired data's influence without full retraining. However, its success depends on the completeness of the identified examples and the quality of the corrective data. In this work, we focus on parameter optimization-based unlearning techniques in the context of code generation tasks.

2.2 Machine Unlearning for Code Generation

A review of related work on MU for code generation tasks reveals only a limited body of research. In prior studies, GA [18–20], which relies solely on the forget objective, has been predominantly employed. These studies observed that GA often causes a significant utility drop, commonly referred to as catastrophic forgetting. To mitigate this issue, subsequent approaches incorporated retain objectives into the unlearning process, leading to the evaluation of techniques such as GA combined with GD [18] and GA combined with KL [18]. Other explored approaches include Direct Preference Optimization (DPO), where data to be forgotten is paired with desired behaviors [19], and NPO [19], which mitigates the catastrophic forgetting associated with GA

by adaptively down-weighting forget-set elements that have already been forgotten. Additional methods include PROD [19], a probability redistribution method that suppresses unwanted tokens. CodeEraser [20] further combines data filtering with optimization-based unlearning by separating sensitive fragments from surrounding code and applying GA, GD, and KL objectives to remove secrets while preserving surrounding code context.

Despite this methodological variety, two aspects of the unlearning process remain largely unexamined. First, existing studies treat the forget and retain sets as fixed inputs rather than as design choices. The order of forget and retain optimization and the way each set is composed are not investigated, even though both can affect the forgetting–utility balance. Second, while individual works target PII and hardcoded secrets within code [18, 20], or whole copyrighted files, vulnerable segments, and deprecated API calls [19], no study contrasts isolated string artifacts embedded in code with semantically coherent program units such as functions and classes under a common method and evaluation. This thesis addresses both gaps directly by treating set dynamics and content type as central factors in unlearning behavior.

2.3 Evaluation of Code Unlearning

Existing work on code unlearning employs a broad range of metrics to evaluate both unlearning effectiveness and model utility. For utility evaluation, HumanEval [2] is among the most widely adopted benchmarks and has been used across numerous studies [18, 20, 24]. It includes self-contained Python functions designed to assess general coding ability. Functional correctness tests are also common for measuring the coding capabilities a model acquires through fine-tuning [18]. Beyond functionality-based evaluation, prior work also relies on similarity-based indicators of utility retention, such as BLEU [25], which captures word-sequence overlap between generated and reference code, and chrF [26], which measures character-level overlap, as well as direct measurements based on cross-entropy loss and perplexity over elements that should remain within the model’s behavior [18, 19, 27].

Unlearning effectiveness is frequently assessed using metrics that also appear in utility evaluation, including BLEU and perplexity. In addition, several studies adopt metrics targeting the leakage of sensitive information within code snippets [18, 27]. The Min-k% membership inference attack [28] is commonly used to determine whether a given data point was present in a model’s training data, while other work evaluates robustness under adversarial conditions, such as prompt injection attacks, to assess whether unlearning techniques remain effective against malicious prompts [19, 24]. A further class of metrics quantifies memorization directly, including Memorization Accuracy (MA) [29] and Extraction Likelihood (EL) [30]. Following this literature, we combine similarity metrics, functional correctness tests, and general code-generation benchmarks. Specifically, we use BLEU and chrF to measure similarity on forget and retain splits, functional correctness tests to evaluate fine-tuning-acquired capabilities, and HumanEval to assess general code-generation utility.

2.4 Dataset Curation for Machine Unlearning

Dataset curation is a fundamental first step in MU tasks and directly shapes unlearning targets. Unlike standard training, where data is typically used for a single optimization objective, unlearning divides data according to its role: the forget set contains elements targeted for removal, while the retain set contains elements whose behavior should be preserved [10]. Prior work selects unlearning targets either by identifying knowledge that models have memorized and can reproduce, or by synthetically introducing such knowledge through fine-tuning before unlearning. Synthetic

dataset curation with fine-tuning provides greater control and helps avoid contamination from pretraining data [31]. However, this scenario is also less realistic, creating a trade-off in dataset design. In the context of code unlearning, prior work has constructed forget sets from code files labelled as copyrighted, deprecated APIs, code snippets vulnerable to cybersecurity attacks, and detected memorized secrets [19, 20, 24, 27]. These code artifacts and secrets are typically selected from existing datasets or scraped from public GitHub repositories.

Maini et al. introduced TOFU [31], one of the most widely used evaluation frameworks for natural language unlearning. TOFU consists of fictitious authors synthetically generated by GPT-4 [32], after which models are trained on this information. This design introduces new knowledge solely through the fine-tuning stage, without overlap with the model’s pretraining data, thereby enabling a cleaner evaluation of unlearning. The fictitious authors are divided into forget and retain sets, together with an in-domain set containing real authors and general knowledge. Some prior work in code unlearning has attempted to follow this design by injecting Personally Identifiable Information (PII) into code snippets [18]. However, this scenario captures only one dimension of code unlearning and overlooks critical cases such as copyrighted code removal. Motivated by this limitation, we introduce a dataset based on synthetic functions and classes with injected secrets. This enables evaluation beyond short-string memorization by incorporating semantically structured code units as unlearning targets.

Methodology

This section describes the methodology used to evaluate machine unlearning for code generation tasks, including dataset construction, model selection, unlearning objectives, and evaluation metrics.

3.1 Research Questions

- **RQ1: What trade-offs emerge when applying different machine unlearning techniques to LLMs for code generation tasks?**

This research question aims to compare multiple MU techniques, such as GA, NPO [19], PROD [19] and their variants that utilize KL and GD retain objectives. Only a subset of these techniques have been applied to LLMs for code generation tasks only in a limited number of studies [18–20]. The objective is to examine how these approaches affect the balance between reducing sensitive data leakage while maintaining overall code generation quality and operating with reasonable energy consumption. We will evaluate both the general capabilities of the model and the retention of fine-tuned functionality after the unlearning process in order to capture model utility. This analysis will guide the selection of a favorable balance for MU in LLMs for coding tasks.

- **RQ2: How does machine unlearning behavior differ between isolated secrets and semantically structured code units?**

Existing MU techniques are designed primarily for natural-language problems, and it is unclear whether they transfer to structured code units, which we define as complete functions or classes that form semantically and syntactically coherent components of a program. This is critical to understand because a function or class is a coherent semantic unit rather than an isolated string (such as API keys, passwords, and email addresses), and forgetting it may require extra attention and data processing steps. If existing techniques degrade or behave differently in this setting, their reported trade-offs may not hold for copyrighted code-removal scenarios. Therefore, this research question tries to answer whether MU techniques can unlearn whole functions and classes, and how the resulting trade-offs differ from the secret unlearning setting.

- **RQ3: How do forget/retain set construction and optimization order influence machine unlearning trade-offs for code generation tasks?**

The purpose of this research question is to examine whether the construction of retain set and execution order of optimizations can affect the overall process and improve the trade-offs between unlearning effectiveness, utility retention, and energy consumption. Forget

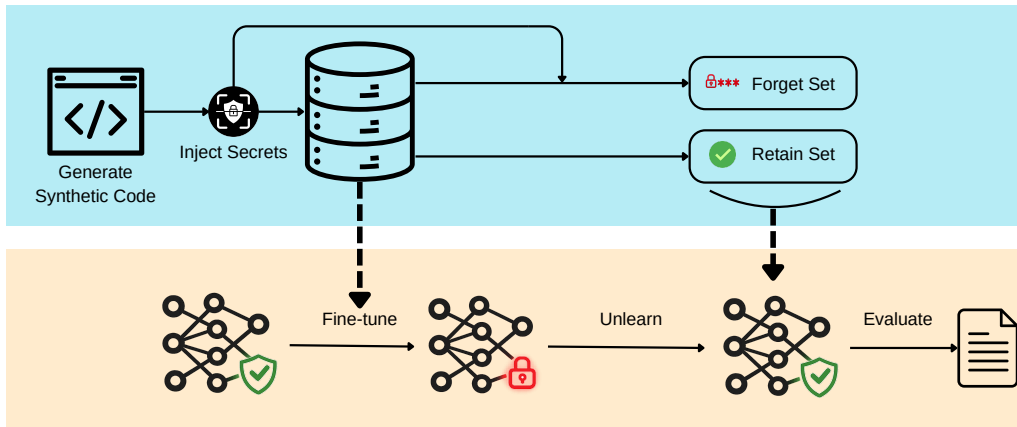


Figure 3.1: Overview of the experimental pipeline. Synthetic code is generated and injected with secrets, then split into forget and retain sets. A base model is fine-tuned on this data, unlearned with the forget/retain sets, and evaluated.

and retain sets are critical for MU tasks, as they guide the model on which elements should be removed and which should be preserved in its behavior. However, to the best of our knowledge, there is no existing work in the literature that examines how the characteristics of these sets affect MU tasks for code. To address this question, we will explore the execution order of the retain and forget objectives, as well as the content and size of the retain set. Through the results, we aim to highlight the importance of data selection and the execution process, in addition to the optimizers used.

3.2 Study Design

In this section, we present the design of our study. The workflow can be seen in Figure 3.1 where we fine-tune pre-trained LLMs on the synthetic dataset, apply each unlearning method under various configurations, and evaluate each resulting model along forgetting effectiveness, retain utility, general code-generation performance, and energy consumption.

3.2.1 Dataset

The dataset is composed of functions and classes, two widely used semantic units in code datasets [33–35]. Because the pre-training corpora of publicly released LLMs are typically unavailable [36], maintaining control over the knowledge a model acquires during fine-tuning by minimizing contamination with pre-training data is essential. We therefore generate these code pieces synthetically using GPT-5.5 [37] and randomize all unit names, so that the exact token sequences are highly unlikely to appear in any existing corpus. As a sanity check, we evaluate the base models

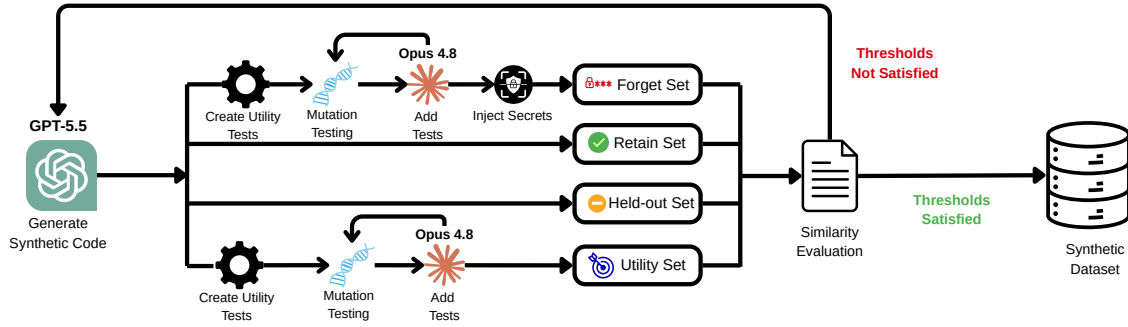


Figure 3.2: Overview of the synthetic dataset construction pipeline. GPT-5.5 generates candidate functions and classes, secrets are then planted into the forget set. All candidates pass through a similarity evaluation, elements that violate the configured similarity thresholds are regenerated.

before fine-tuning and confirm that they do not confidently complete the planted secrets or code units. The primary objective is to assess the effectiveness and utility retention of unlearning directly against the fine-tuning dataset: reverting the model’s behavior to its baseline on the forget set while preserving the retain set and utility. This requirement is the driving motivation for using synthetically generated code. Figure 3.2 gives an overview of the dataset construction pipeline, from synthetic generation through similarity-based evaluations. We explain each stage in detail in the following subsections.

Dataset Structure

The dataset comprises four components: a retain set, a forget set, a general utility set, and a held-out set, containing 300, 150, 100, and 50 code snippets respectively. Each of these sets is divided equally between functions and classes (see Tables 3.1 and 3.2). The forget and retain sets are the two driving sets for the unlearning and retention objectives. The held-out set contains functions and classes that do not contribute to the unlearning process. The utility set is used to measure the effect of unlearning on fine-tuning-acquired utility.

Table 3.1: Overview of the dataset composition. Each component is divided equally between functions and classes.

Component	Functions	Classes	Total	Role
Retain set	150	150	300	Retention objective
Forget set	75	75	150	Unlearning objective
Utility set	50	50	100	Fine-tuning utility effect
Held-out set	25	25	50	In-domain, not unlearned or retained
Total	300	300	600	

The forget set differs from other sets since we inject fake secrets and PII into functions and classes. These sensitive strings are placed either in the documentation immediately following the signature or as a variable assignment within the code. To establish a controlled comparison between code-unit and secret unlearning to answer **RQ2**, the forget set varies across experimental

Table 3.2: Composition of the forget set by code-unit complexity and injected-secret type.

Complexity	Functions	Classes	Total
Simple	25	25	50
Moderate	25	25	50
Complex	25	25	50
Total	75	75	150
Injected secrets: 50 API keys, 50 passwords, and 50 emails			

settings. The secret forget set contains only the raw sensitive content, whereas the code-unit forget set contains the entire functions and classes in which these secrets appear. Examples of a forget function and class are shown in Listings 3.1 and 3.2.

```
def synthetic_forget_function_UAKw9i(values):
    """Convert integers into sorted inclusive ranges.

    Duplicate values are ignored. Empty input returns
    an empty list. Each range is returned as a
    (start, end) tuple, where single values have
    equal ends.

    Example: synthetic_forget_function_UAKw9i(
        [4, 2, 3, 8]) -> [(2, 4), (8, 8)]
    email_EU1jjQ = "kxkdp@example.com"
    """
    nums = sorted(set(values))
    if not nums:
        return []
    ranges = []
    start = nums[0]
    previous = nums[0]
    for value in nums[1:]:
        if value == previous + 1:
            previous = value
        else:
            ranges.append((start, previous))
            start = previous = value
    ranges.append((start, previous))
    return ranges
```

Listing 3.1: Synthetic forget function with injected email.

```

class synthetic_forget_class_NbrnTP:
    '''A small lending shelf tracks copy counts and
    one active loan per member.

    Public methods:
    api_key_3fAbnF = "sk-ISRABr1VjArnVgxwvqcCh5esM2wleOV9"
    - borrow_copy(member, title): loan a title if a
      copy is available and the member has no active loan.
    - shelve_return(member, days_late=0): return the
      member's loan and report any late fee.

    Example:
    shelf = synthetic_forget_class_NbrnTP({"Dune": 1});
    shelf.borrow_copy("ada", "Dune") == "borrowed"'''
    def __init__(self, copies):
        self.copies = dict(copies)
        self.loans = {}

    def borrow_copy(self, member, title):
        if member in self.loans:
            return "member-has-loan"
        if self.copies.get(title, 0) <= 0:
            return "unavailable"
        self.copies[title] -= 1
        self.loans[member] = title
        return "borrowed"

    def shelve_return(self, member, days_late=0):
        if member not in self.loans:
            return {"accepted": False, "fee": 0}
        title = self.loans.pop(member)
        self.copies[title] = self.copies.get(title, 0) + 1
        fee = 0
        if days_late > 0:
            fee = days_late * 25
        return {"accepted": True, "title": title, "fee": fee}

```

Listing 3.2: Synthetic forget class with injected API key.

Functions and classes are further categorized by complexity, computed via cyclomatic complexity (CC): (i) over the whole body for functions, and (ii) as the average CC across methods for classes. For classes, we exclude `__init__` methods, since in our synthetic code they handle variable assignment rather than business logic. We group code units using the risk bands presented in McCabe's software-quality metrics guidance [38, 39]: simple/low-risk (1–10), moderate-risk (11–20), and complex/high-risk (21–50).

Functional Correctness Tests

The forget and utility sets also include unit-test benchmarks, referred to as ForgetEval and UtilityEval, respectively, that verify the functionality of each code unit. These tests are generated by GPT-5.5 during synthetic code generation. Since mutation analysis is considered one of the strongest indicators of test-suite adequacy [40, 41], we add a mutation-testing step to increase our test coverage. This process is based on altering the code slightly to produce variants called mutants, and we check whether the tests detect the resulting change. A mutant is said to be killed if at least one test fails on it, indicating that the suite has sufficient coverage to detect that anomaly. We run mutation testing with `mutmut` [42], a mutation-testing tool for Python. We then provide the mutation report to Claude Opus 4.8¹, which generates additional tests. Using a different model

¹<https://www.anthropic.com/news/claude-opus-4-8>

for this hardening step reduces the likelihood that the additional tests reproduce the same weaknesses as the initial GPT-5.5-generated tests [43]. This raises the mutation score from 53.7% to 79.4% for the forget set and from 79.2% to 93.9% for the utility set unit tests. Finally, we revalidate that every solution passes its own tests.

Diversity

A key concern when generating code with an LLM is the diversity and uniqueness of the resulting samples. We assess this along lexical, structural, and semantic similarity dimensions. Lexical similarity is the Jaccard overlap of the token IDs produced by the `gpt-oss-120b` tokenizer. Structural similarity is measured with CodeBLEU under the default equal-weight configuration (n -gram, weighted n -gram, syntax, and dataflow matches, each = 0.25) [44]. Because CodeBLEU is asymmetric, we average the forward and reverse scores for each unordered pair. Semantic similarity is measured with code embedding models, validated across two independent models (SFR-Embedding-Code-400M_R² and Qodo-Embed-1-1.5B³). Transformer-based embedding models are known to compress their outputs into a narrow region of the space, making arbitrary pairs appear similar [45], which is called anisotropy. To mitigate this drawback of transformer-based embedding models, we mean-center each embedding before comparison [46].

Across the whole dataset, we treat 0.6 as a maximum similarity acceptance threshold for all three metrics, rewriting or regenerating any code that violates it. This is evaluated over all 179,700 pairs of the 600 samples. We adopt 0.6 with metric-specific justification. For semantic (embedding) similarity, 0.6 is an established cosine-similarity threshold for flagging content that is related, but not identical [47], and recent calibration of cosine similarity against empirical semantic-duplicate rates places the decision boundary in the 0.5–0.6 range [48]. For token similarity, large-scale code-deduplication pipelines typically use a looser threshold of 0.7 [49, 50] so our 0.6 cut is stricter, aiming to bias construction toward well-separated examples.

Evaluation reveals that no pair exceeds 0.6 on any of the three measures after the regeneration process. CodeBLEU and token Jaccard similarity both concentrate on low values (Figure 3.3). The most-similar pairs reach only 0.555 CodeBLEU score and 0.534 token Jaccard similarity. The structural dimension is most evident in the closest CodeBLEU pair, where two synthetic classes show strong syntactic similarity (0.69) and dataflow similarity (0.83), despite weak n -gram overlap (0.34). This suggests that CodeBLEU captures structurally similar code that lexical overlap alone would likely treat as irrelevant. We therefore apply the threshold to the aggregate CodeBLEU score rather than to each individual component. On the semantic axis, the mean-centered cosine is concentrated near zero and reaches the 0.6 threshold for no pair under either model. The contrast with the raw cosine quantifies the anisotropy directly: under raw cosine, the mean pairwise similarity is 0.57–0.59 and between 28% (Qodo) and 40% (SFR) of all pairs exceed 0.6, whereas after mean-centering that fraction is exactly zero (Figure 3.4).

3.2.2 Model Selection

Evaluating multiple types of LLMs is important for understanding how differences in training objectives, data scope, and architecture affect the effectiveness, utility impact, and computational cost of MU techniques. To ensure that the results generalize beyond specialized code models, we evaluate a domain-specific code model and a general-purpose LLM. Specifically, we select two models with comparable sizes from different modern families: Meta Llama 3.2 3B⁴ [51], and the

²https://huggingface.co/Salesforce/SFR-Embedding-Code-400M_R

³<https://huggingface.co/Qodo/Qodo-Embed-1-1.5B>

⁴<https://huggingface.co/meta-llama/Llama-3.2-3B>

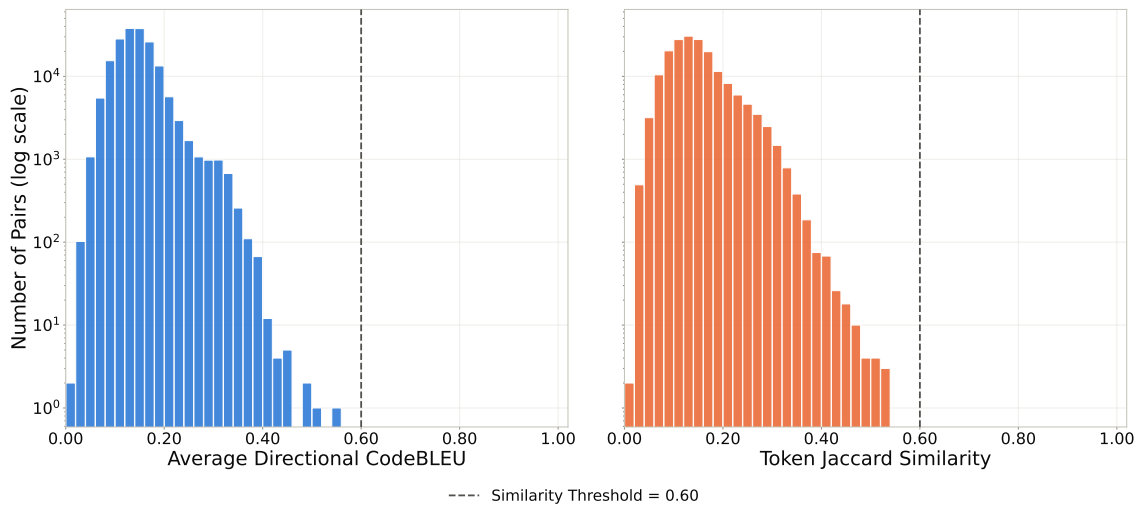


Figure 3.3: Pairwise lexical and structural similarity over all 179,700 pairs of the final dataset (log-scaled counts). Left: direction-averaged CodeBLEU; Right: token Jaccard similarity. Both lie entirely below the 0.6 threshold (dashed red), with maxima of 0.555 and 0.534 respectively

widely adopted code-specialized Qwen2.5 Coder 3B⁵ [52].

3.2.3 Unlearning Objectives

In this section, we present the unlearning objectives evaluated in this work, grouped into two categories according to their goal: forgetting and retaining. These objectives are combined exhaustively in order to identify which combinations are most effective for code unlearning tasks.

Forget Objectives

- **Gradient Ascent (GA)** is a simple and widely adopted unlearning approach in prior work [53]. It performs gradient ascent on the loss of the next-token prediction over forget set elements. The goal is to reverse the gradient descent effect during training on the forget set data. Although it is known to be susceptible to catastrophic forgetting, often degrading model utility to unusable levels, it remains a valuable baseline for comparing unlearning techniques.
- **Negative Preference Optimization (NPO)** is an unlearning method inspired by direct preference optimization, which targets the mitigation of the catastrophic forgetting problem, making it a meaningful point of comparison to the GA forget objective [54]. It achieves this by employing an oracle, the frozen original model, to adaptively down-weight forget-set elements that have already been forgotten, yielding a more stable training process than GA while continuing to suppress remaining memorized patterns [54].
- **PROD** is an unlearning technique that redistributes the probability mass assigned to sensitive tokens over the remaining vocabulary, specifically designed for code unlearning tasks [19]. It has been shown to cause a smaller utility drop than NPO and GA while achieving

⁵<https://huggingface.co/Qwen/Qwen2.5-Coder-3B>

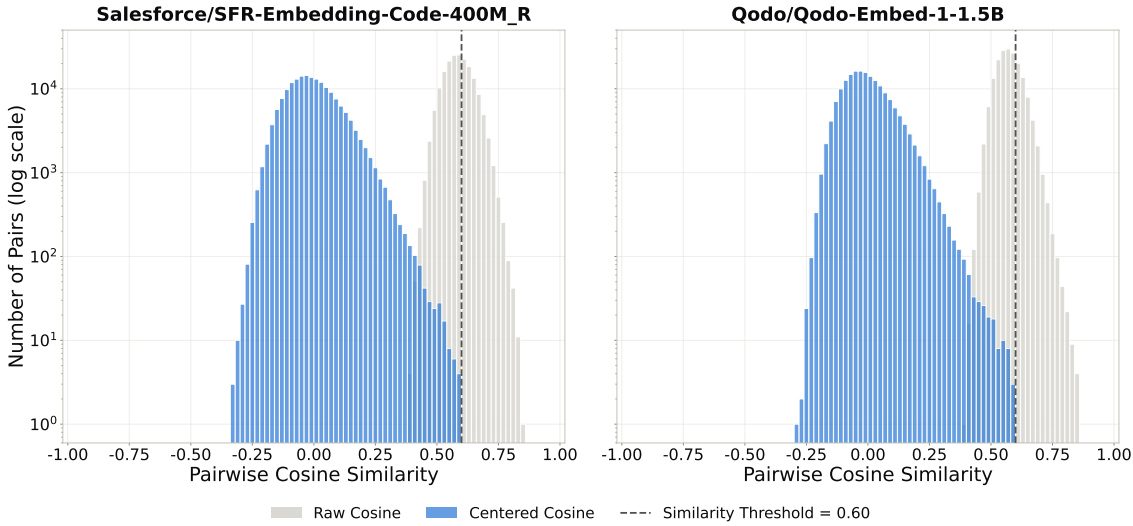


Figure 3.4: Pairwise cosine similarity over all 179,700 pairs for two code embedding models (left: SFR-Embedding-Code-400M_R; right: Qodo-Embed-1-1.5B). Raw cosine (grey) is compressed into a narrow band near 0.6, illustrating embedding anisotropy. After mean-centering (blue), the distribution re-centers near 0.

comparable unlearning effects. However, this method has not previously been evaluated in combination with additional retain objectives. PROD is therefore selected to investigate whether it outperforms other techniques for code unlearning across different objective combinations.

Retain Objectives

- **Gradient Descent (GD)** is the direct counterpart of GA: model parameters are updated by descending the gradient of the loss computed on the retain set, thereby reinforcing the patterns present in the retained elements [55]. It is selected as an oracle-free baseline to update the model weights with the aim of preserving generation capabilities.
- **Kullback–Leibler Divergence (KL)** preserves the output distribution of the unlearned model on the retain set by minimizing the divergence between the unlearned model and the oracle model [56], which provides the reference output distributions on the retain set. Relative to GD, this method introduces additional complexity through the use of an oracle model, but it more closely respects the original model’s behavior. The resulting trade-off between computational overhead and retention quality makes it a useful point of comparison among retain objectives.

3.2.4 Evaluation

Security Game

We evaluate forgetting through a prefix/suffix reconstruction game [57]. The model is given a prefix p from the training data and is prompted to generate the corresponding suffix s . For copyrighted code, the prefix consists of the function or class signature together with its documentation,

while the suffix corresponds to the code unit body. In contrast, for secret unlearning, the prefix is the code snippet before the secret is introduced, and the suffix corresponds directly to the secret value.

Metrics and Tools

To select metrics suited to the secret-unlearning and copyrighted-code scenarios, we first define the goal of unlearning in each case. For secrets, revealing more characters reduces the attacker’s uncertainty and can make attacks easier by lowering the number of guesses required. For instance, prior work on partial-password authentication shows that partial observations can increase attack success [58]. The goal is therefore to minimize overlap between the generated output and the original secret. We measure recovery at the character level using *chrF* [26], which captures how much of the secret reappears in the output. For copyrighted code, intellectual property rights protect expression rather than ideas [59, 60]. This makes textual similarity a relevant signal for copyright-oriented evaluation, since literal reproduction of source-code is more directly related to copyright concerns than functional equivalence alone. Following this we use a text-similarity metric, more specifically *BLEU* score [25], which is also aligned with previous code unlearning work [19]. Both scores are computed with *sacreBLEU* [61] and forget quality is reported as $1 - chrF$ for secrets, and $1 - BLEU$ for code units. As mentioned in the previous section, the utility of fine-tune acquired functionality is measured by UtilityEval benchmark pass rates and energy consumption of unlearning techniques are estimated using CodeCarbon Python library [62].

Experimental Design

In this section, we describe our experimental design decisions and configurations for the unlearning process.

4.1 Fine-Tuning

As an initial prerequisite, we fine-tuned the selected models using Low-Rank Adaptation (LoRA) [63] on synthetic data with the goal of inducing strong memorization of the synthetic dataset. To achieve this, we duplicated each code unit three times, increasing the dataset size from 600 to 1,800 data points. As a result, every individual function and class was seen three times within each epoch. We trained the models with a learning rate of 0.0002, a training batch size of 8, and gradient accumulation of 4 for 5 epochs. We then evaluated the average similarity score between the generated output and the training data on both code units and secrets in the forget set. This was done for both the base models and every epoch of the fine-tuned models, as shown in Figure 4.1. The results indicate that memorization originates from acquired knowledge rather than pretraining, since at epoch 5 the similarity scores increase from the 0.05–0.17 range to 0.83–1.0. We therefore use the checkpoint from epoch 4 for the unlearning experiments, as it is the earliest checkpoint at which both models reach the memorization plateau for both code-unit and secret targets.

To establish a consistent baseline across all elements, we retain only the secrets and code units that the fine-tuned model reproduces exactly in at least one of ten generations, as well as test tasks that the baseline passes in at least one of ten attempts. The retained fractions are shown in Figure 4.2. For secrets, a high proportion is exactly reproduced by both models (86.0% and 90.0%). For code units, the held-out set is fully reproduced, while the retain set achieves nearly identical results ($\geq 96.0\%$). The only exception is the Llama3.23B code-unit forget set, where the retained fraction is 64.7%. Moreover, both models pass 83.3% of the ForgetEval tests and achieve nearly perfect scores on UtilityEval (99.0% and 100.0%). This filtering ensures that every metric starts from a baseline of 1.0, so that subsequent results capture the relative degradation caused by unlearning rather than contaminating it with incomplete baseline memorization or test coverage.

4.2 Unlearning

To answer our research questions systematically, we design multiple experimental setups and organize them according to the research question they address. Before conducting the main experiments, we tune the learning rate for each forgetting objective separately, since different ob-

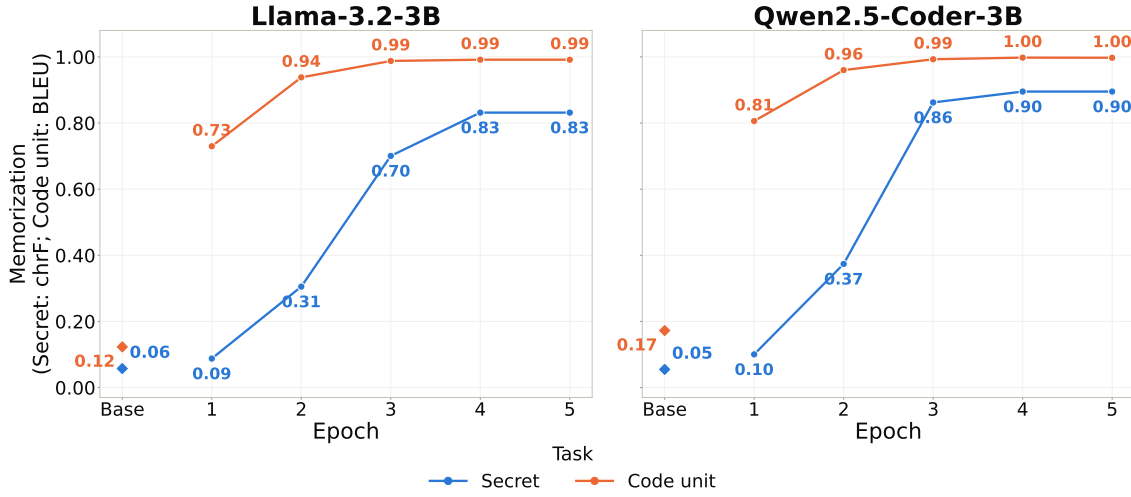


Figure 4.1: Mean generated-target similarity across the base and five training epochs for both models. Secret memorization is measured with chrF and code-unit memorization with BLEU, averaged over 150 examples; higher values indicate greater memorization.

jectives can respond differently to the same learning rate. To this end, we perform a grid search over the three forgetting objectives using the Qwen2.5-Coder-3B model, running each configuration for 3 epochs. For GA and NPO, we use the set $\{1e-4, 5e-5, 1e-5, 1e-6\}$. However, our initial experiments revealed that PROD required higher learning rates to match the forgetting quality of the other techniques. Therefore, for PROD, we modify the search space to $\{1e-4, 3e-4, 4e-4, 5e-4, 5e-5\}$. Based on the results of this preliminary hyperparameter tuning, shown in Figure 4.3, we select the learning rates used in the remaining experiments. The selected unlearning hyperparameters across all setups are summarized in Table 4.1. Unlearning is applied directly to the LoRA adapters obtained from fine-tuning described in section 4.1, rather than to a merged or full-parameter model. All unlearning experiments were executed using a single NVIDIA A100 GPU with 80 GB of memory. The random-order runs used the A100 PCIe variant, whereas the forget-first, retain-first, and double-retain runs used the A100 SXM4 variant.

RQ1: Trade-offs of unlearning techniques. As discussed in the previous sections, we use NPO, GA, and PROD as forget objectives, and KL and GD as retain objectives, combining them exhaustively. This results in three forget-only methods and six combined methods, yielding nine experimental setups in total, as shown in Table 4.2. We use secret unlearning as the baseline for this research question.

RQ2: Secret vs. code-unit unlearning. To answer this research question, we apply the same unlearning techniques in the code-unit unlearning setup and compare the results to assess the transferability of unlearning techniques to semantically structured units. By keeping the method configurations fixed across both settings, we isolate the effect of the unlearning target type.

RQ3: Retain set characteristics. To analyze the effects of retain and forget objective dynamics on MU trade-offs, we vary several characteristics of the experimental configuration across three dimensions: (i) the ordering strategy between forgetting and retain optimization, (ii) the content of the retain set, and (iii) the relative size of the retain set compared to the forget set. The resulting

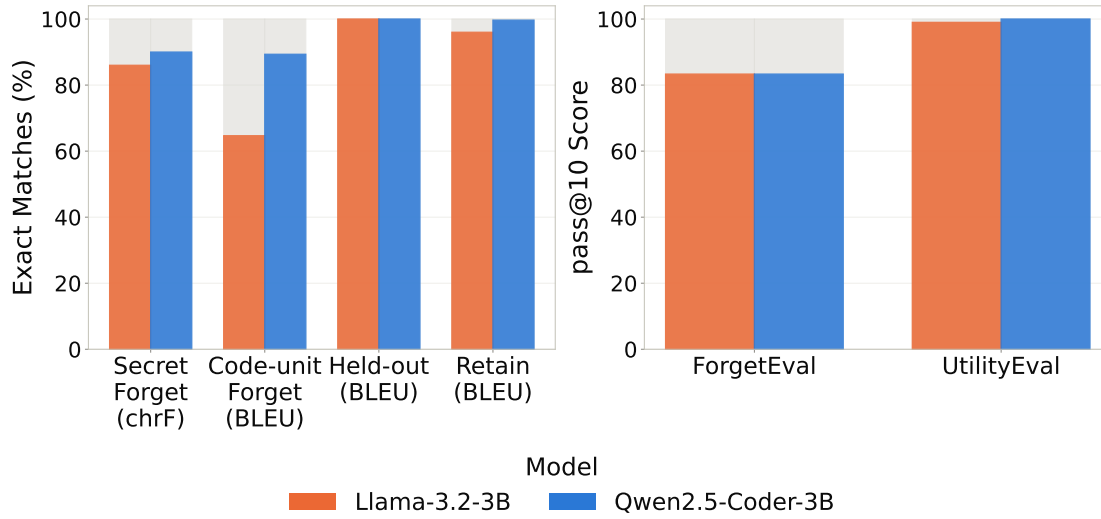


Figure 4.2: Fine-tuned models checkpoint four performance before unlearning. The left panel shows the percentage of items with at least one exact match among ten generations, while the right reports ForgetEval and UtilityEval pass@10 scores.

experimental configurations are summarized in Table 4.3. Rather than exhaustively evaluating the Cartesian product of all dimensions in Table 4.3, we adopt a controlled ablation design. In each experiment, we vary only one dimension at a time and keep the remaining dimensions fixed to a default configuration. This design enables a more direct interpretation of how each individual factor affects the trade-off between forgetting and retention. Unless stated otherwise, the default configuration uses mixed ordering, excludes functional correctness test targets from the retain set, and sets the retain set size equal to the forget set size. This is also the configuration used in the **RQ1** experiments. To reduce the experimental space under resource constraints, we select the best performing retain objective according to the forgetting–utility trade-off for each forget objective based on the **RQ1** results.

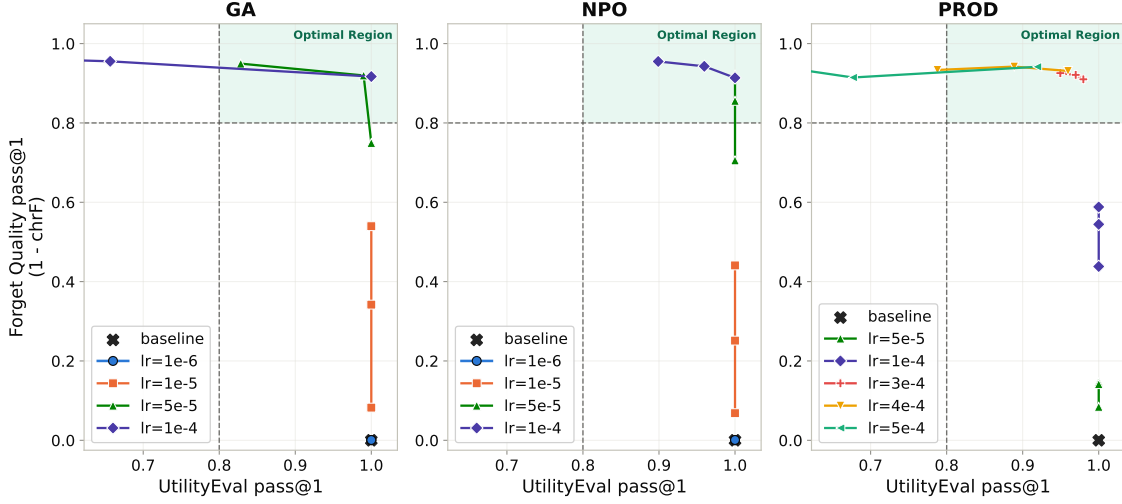


Figure 4.3: Trade-off between forgetting quality and model utility across learning rates for GA, NPO, and PROD. The y-axis shows the reduction in average similarity score relative to the baseline model. The x-axis shows UtilityEval retention relative to the baseline. Each line corresponds to a learning rate, with points representing different epochs. The shaded top-right region indicates the preferred optimal area, where models achieve high forgetting effect while retaining utility.

Table 4.1: Unlearning configuration. The combined objective is $\mathcal{L} = \mathcal{L}_{\text{forget}} + \lambda_{\text{ret}} \mathcal{L}_{\text{retain}}$. Values marked $\dagger$ are confirmed against the respective source implementation.

Setting	Value	Notes
<i>Shared</i>		
Base checkpoint	FT epoch 4 (per model)	memorization plateau
Oracle model	frozen 8-bit quantized FT epoch 4	used by NPO, KL, PROD
Optimizer	AdamW	
Effective batch size	32	
Precision	bf16	
Epoch	3	
<i>Forget objectives</i>		
GA learning rate	5e-5	
NPO learning rate	1e-4	
NPO β	0.1 †	
PROD learning rate	3e-4	
PROD α	0 †	
PROD	0.8 †	
<i>Retain objectives</i>		
Retain weight λ_{ret}	1.0	GD and KL

Table 4.2: Evaluated unlearning techniques.

Forget Objective	Retain Objective
GA	–
GA	GD
GA	KL
NPO	–
NPO	GD
NPO	KL
PROD	–
PROD	GD
PROD	KL

Table 4.3: Experimental configurations for analyzing retain set size and forget-retain objective ordering for MU.

Dimension	Configuration	Description
Ordering strategy	Forget $\rightarrow$ Retain	Forget-set optimization is performed first, followed by retain-set optimization.
	Retain $\rightarrow$ Forget	Retain-set optimization is performed first, followed by forget-set optimization.
	Random ordering	Each batch contains either forget or retain samples, and the two batch types are processed in a random order during optimization.
Retain-set size	Equal to forget set	The retain set contains the same number of samples as the forget set.
	Double the forget set	The retain set contains twice as many samples as the forget set.

Results

This section presents and analyzes the experimental results to answer the three research questions.

5.1 RQ1: Machine Unlearning Technique Trade-offs

RQ1 investigates how different MU techniques affect forgetting quality, utility retention, and resource consumption. To answer this question, we evaluate each technique separately on two models, on the task of unlearning secrets embedded in functions and classes.

GA-based techniques As shown by the GA rows of Table 5.1, forget quality improves with additional epochs without substantial utility degradation with final epoch for plain GA retaining 71.7% (Llama) and 90.9% (Qwen) of baseline utility. Interestingly, adding a retain objective on top of GA accelerates the collapse at earlier epochs. For instance, at epoch 2, plain GA still retains 100.0% (Llama) and 99.0% (Qwen) of utility, whereas GA+KL retains only 33.3% and 66.7%, respectively, and by epoch 3 the retain objective variants fall as low as 23.2% (Llama) and 43.4% (Qwen). GA-based techniques are therefore only competitive early in the unlearning process. On both models, the most favorable point occur at epoch 1 for GA+GD and GA+KL, and at either epoch 2 or epoch 3 for plain GA, where they combine forget quality of at least 83.2% with at most 2.0% utility loss.

NPO-based techniques. In contrast to GA, NPO provides a more stable and predictable trade-off, as shown by the NPO rows of Table 5.1. With a retain objective, every NPO variant loses at most 2.0% utility across all epochs, underlining the benefit of adaptive weighting of NPO over GA. The exception to this stability is NPO without a retain objective on Llama 3.2 3B, whose utility collapses to 26.3% at epoch 3. The two retain objectives behave similarly on Llama 3.2 3B, where the GD and KL variants yield nearly identical trade-offs (forget quality 89.9–90.8% at 98.0–100.0% utility). On Qwen 2.5 Coder, the KL variant achieves clearly higher forget quality (86.4–87.1% vs. 75.3–80.2% for GD), while its utility retention is comparable (at least 98.0% vs. 99.0% for GD). Base NPO can provide a balanced trade-off, but its behavior is not consistent across the two models.

PROD-based techniques. The PROD rows of Table 5.1 show that plain PROD provides a more stable trade-off than its retain-objective variants. It reaches at least 83.8% forget quality at every epoch on both models while retaining at least 94.9% utility. When a retain objective is added, the KL variant tends to retain more utility than GD at the same epochs, while the forget quality of the two variants differs by at most 2.9 percentage points. Overall, adding a retain set still destabilizes

PROD at later epochs: by epoch 3, PROD+GD retains only 48.5% (Llama) and 77.8% (Qwen) of baseline utility, a drop of up to 46.4 percentage points relative to plain PROD.

Table 5.1: Secret unlearning results at every epoch. Each result cell reports Forget Quality / UtilityEval, where forget quality uses 1 – chrF.

Model	Technique	pass@1			pass@5			pass@10		
		Epoch 1	Epoch 2	Epoch 3	Epoch 1	Epoch 2	Epoch 3	Epoch 1	Epoch 2	Epoch 3
Llama-3.2-3B	GA	0.863 / 0.994	0.927 / 0.908	0.981 / 0.507	0.817 / 1.000	0.880 / 0.992	0.971 / 0.673	0.802 / 1.000	0.865 / 1.000	0.969 / 0.717
	GA+GD	0.927 / 0.918	0.963 / 0.441	0.973 / 0.108	0.882 / 0.988	0.960 / 0.637	0.969 / 0.239	0.866 / 0.990	0.959 / 0.687	0.969 / 0.293
	GA+KL	0.931 / 0.911	0.972 / 0.093	0.975 / 0.086	0.898 / 0.987	0.969 / 0.248	0.970 / 0.182	0.879 / 0.990	0.968 / 0.333	0.969 / 0.232
	NPO	0.931 / 0.968	0.967 / 0.391	0.996 / 0.105	0.899 / 1.000	0.948 / 0.632	0.992 / 0.216	0.882 / 1.000	0.943 / 0.697	0.991 / 0.263
	NPO+GD	0.935 / 0.930	0.937 / 0.934	0.934 / 0.918	0.912 / 0.975	0.916 / 0.988	0.910 / 0.988	0.899 / 0.980	0.908 / 1.000	0.901 / 1.000
	NPO+KL	0.929 / 0.954	0.934 / 0.944	0.935 / 0.917	0.906 / 0.987	0.914 / 0.999	0.917 / 0.978	0.899 / 0.990	0.906 / 1.000	0.905 / 0.980
	PROD	0.955 / 0.898	0.945 / 0.784	0.947 / 0.687	0.925 / 0.991	0.902 / 0.967	0.890 / 0.913	0.912 / 1.000	0.879 / 0.980	0.863 / 0.949
	PROD+GD	0.938 / 0.625	0.932 / 0.393	0.923 / 0.195	0.881 / 0.872	0.869 / 0.660	0.851 / 0.395	0.846 / 0.899	0.852 / 0.727	0.831 / 0.485
	PROD+KL	0.941 / 0.849	0.913 / 0.631	0.934 / 0.345	0.878 / 0.964	0.852 / 0.891	0.859 / 0.641	0.857 / 0.980	0.823 / 0.960	0.832 / 0.737
Qwen2.5-Coder-3B	GA	0.791 / 0.986	0.909 / 0.942	0.948 / 0.777	0.714 / 0.990	0.867 / 0.986	0.927 / 0.885	0.679 / 0.990	0.832 / 0.990	0.914 / 0.909
	GA+GD	0.934 / 0.931	0.950 / 0.609	0.971 / 0.396	0.879 / 0.987	0.926 / 0.706	0.957 / 0.548	0.857 / 1.000	0.911 / 0.737	0.952 / 0.586
	GA+KL	0.910 / 0.927	0.961 / 0.522	0.977 / 0.252	0.853 / 0.975	0.939 / 0.627	0.968 / 0.399	0.846 / 0.980	0.933 / 0.667	0.964 / 0.434
	NPO	0.907 / 0.982	0.942 / 0.912	0.950 / 0.830	0.855 / 0.990	0.919 / 0.978	0.930 / 0.913	0.838 / 0.990	0.911 / 0.990	0.918 / 0.929
	NPO+GD	0.833 / 0.970	0.857 / 0.937	0.894 / 0.963	0.776 / 0.995	0.785 / 0.982	0.827 / 0.990	0.753 / 1.000	0.763 / 0.990	0.802 / 0.990
	NPO+KL	0.928 / 0.982	0.931 / 0.967	0.917 / 0.979	0.897 / 0.995	0.886 / 0.980	0.893 / 0.990	0.869 / 1.000	0.871 / 0.980	0.864 / 0.990
	PROD	0.918 / 0.968	0.941 / 0.924	0.931 / 0.844	0.874 / 0.988	0.868 / 0.987	0.878 / 0.963	0.838 / 0.990	0.843 / 0.990	0.845 / 0.970
	PROD+GD	0.912 / 0.858	0.923 / 0.596	0.939 / 0.459	0.835 / 0.972	0.865 / 0.847	0.884 / 0.704	0.796 / 0.980	0.833 / 0.929	0.849 / 0.778
	PROD+KL	0.903 / 0.884	0.934 / 0.757	0.946 / 0.641	0.835 / 0.950	0.882 / 0.927	0.895 / 0.863	0.817 / 0.960	0.847 / 0.960	0.878 / 0.899

Cross-technique comparison. To compare techniques against each other, we select the epoch that maximizes an equal-weight score (the mean of forget quality and utility score) for each technique, where both forget quality and utility score reach at least 0.80. This isolates the region where forgetting is effective while utility is largely preserved. All nine techniques qualify for each model, as shown in Figure 5.1. For Llama 3.2 3B, base PROD without a retain objective achieves the highest score (0.956, with 91.2% forget quality at unchanged utility) and Pareto-dominates every other selected configuration, followed closely by NPO+GD and NPO+KL (0.953–0.954), which likewise preserve utility fully. For Qwen 2.5 Coder, plain NPO ranks first (0.951, with 91.1% forget quality at 99.0% utility). NPO-based techniques are, however, less uniform on Qwen 2.5 Coder than on Llama 3.2 3B: NPO+KL still ranks second (0.935), but NPO+GD performs the lowest forget quality among all selected configurations (80.2%), only slightly above the qualification threshold. GA-based techniques remain comparable in score (0.911–0.935); notably, on Qwen 2.5 Coder, plain GA at epoch 3 achieves the highest forget quality of all selected configurations (91.4%) but at the lowest utility (90.9%), placing it on the Pareto frontier alongside NPO and NPO+KL. Overall, plain PROD provides the strongest peak trade-off on Llama 3.2 3B, plain NPO on Qwen 2.5 Coder, and the NPO retain-objective variants provide the most consistently stable performance. Across both GA and PROD, retain objectives do not reliably improve the trade-off.

To verify that the findings are not specific to the selected minimum thresholds of 0.80 for UtilityEval and forget quality, we conduct two sensitivity analyses. We first vary both thresholds jointly and then vary each threshold independently while fixing the other at 0.80. The complete results are reported in Appendix Tables 1 and 2, respectively. The selected epochs and Pareto frontier remain stable across a broad range of thresholds, although stricter forget quality thresholds substantially reduce method coverage. This supports 0.80 as a balanced cutoff that preserves both performance requirements and representative coverage. To further assess the robustness of this selection procedure, we examine whether baseline exact match and UtilityEval filtering influences the reported results. We repeat the best-epoch selection using the unfiltered evaluation sets while retaining the same 0.80 UtilityEval and forget quality thresholds. Findings indicate that filtering has little effect on overall coverage: 35 of 54 checkpoints qualify with filtering compared with 36 without filtering, and all 18 model–technique configurations remain eligible in both cases. The selected epoch is unchanged for 15 of 18 configurations, while three of the four filtered Pareto-optimal points remain on the unfiltered frontier (see Appendix Table 3). Across all checkpoints, filtering changes UtilityEval by only (+0.12) pp and forget quality by (−0.22) pp on average.

Resource consumption. We compare average GPU energy utilized over both models to analyze the resource consumption. From Figure 5.2 it can be seen that forget objectives alone consume less energy than their retain-objective variants in every forget family, with GA requiring the least amount of resources (0.47 Wh). Another observation is that KL consistently consumes more energy than the GD retain objective in every family and on both models (+31% for NPO, +55% for PROD, and +68% for GA on average), most likely due to the KL requiring forward passes through a frozen oracle model. Interestingly, NPO+KL achieves the lowest GPU energy consumption among the KL variants (3.12 Wh versus 3.28 Wh for GA+KL and 3.72 Wh for PROD+KL), making it slightly more efficient. The KL overhead is large enough that GA+KL costs more than NPO+KL, erasing plain GA’s substantial baseline advantage. It should be noted that all these values are negligible compared to retraining a model from scratch; however, considering the increasing size of modern model families and unlearning being a continuous process, the difference between techniques matters in the long run.

Utility around secrets. The secret-unlearning setup aims to prevent models from leaking sensitive information without affecting the functionality that surrounds these values. To test whether unlearning side effects spill over into the neighbouring code, we use ForgetEval, which runs

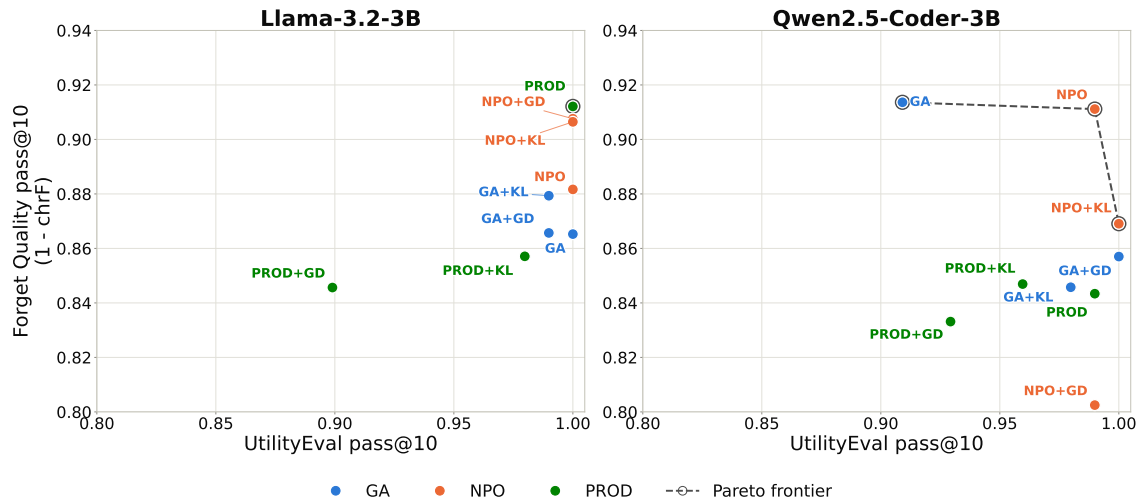


Figure 5.1: Utility–forgetting trade-off for the selected secret-setting checkpoints at pass@10. For each model–technique configuration, we consider epochs satisfying the minimum thresholds of 0.80 for both UtilityEval and forget quality and select the epoch maximizing their equally weighted mean. Dark outlined markers and dashed lines identify the model-specific Pareto frontiers.

functional correctness tests on the forget-set functions and classes into which the secrets were injected. Results in Table 5.2 show that GA-based techniques with retain objectives cause the most collateral damage, driven almost entirely by Llama 3.2 3B: GA+GD and GA+KL fail 24.0% and 19.2% of the previously passing tests on Llama 3.2 3B, compared to only 1.6% and 3.2% on Qwen 2.5 Coder. NPO-based retain variants preserve the surrounding utility best, with NPO+KL achieving the highest mean ForgetEval score (0.996), followed by NPO+GD (0.988), making these two techniques highly suitable for preserving code adjacent to the secret being removed. Outside the GA family, plain NPO and PROD+GD are the weakest configurations (0.924 each). We next examine whether the location of the injected secret affects ForgetEval failures. Figure 5.3 shows that functions and classes whose secret is planted in the documentation fail at a far lower rate than units whose secret is embedded in the implementation. Aggregated over all techniques at pass@10, 11.3% of evaluations of code-located secrets fail, compared to 2.2% for documentation-located secrets. At pass@1 the gap is even wider (45.8% vs. 12.3%).

We verify this association separately for each of the 18 technique–model combinations using a two-sided Fisher exact test. Because 18 model–technique combinations were tested at each pass@ k , Benjamini–Hochberg correction was applied within each pass@ k to control the false discovery rate. After correction, secret location was significantly associated with ForgetEval failure in 16 of 18 combinations at pass@1, 9 of 18 at pass@5, and 5 of 18 at pass@10. The declining number of significant associations at larger k is expected because additional generations provide more opportunities to produce a passing solution. Consequently, fewer tasks remain failed at pass@5 and pass@10. Therefore, the decline does not necessarily indicate that secret location becomes irrelevant; rather, persistent failures become less frequent and harder to distinguish statistically.

Takeaway: When robustness matters most, NPO+KL is the safest default: it is the only configuration with stable forget quality and near-baseline utility across all epochs and both

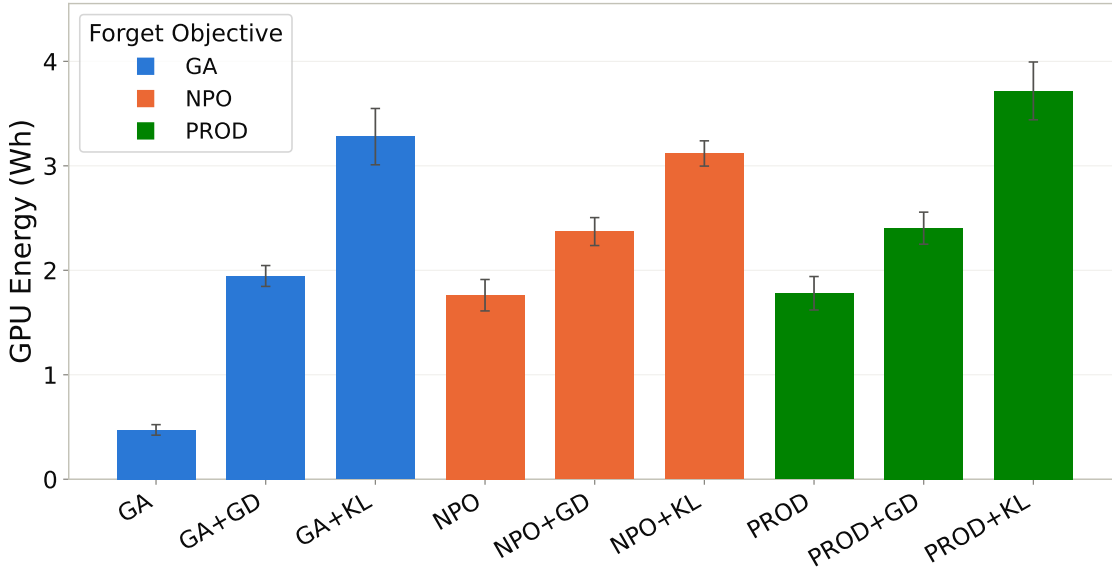


Figure 5.2: Measured GPU energy for secret-setting unlearning runs. Bars show the mean across Llama-3.2-3B and Qwen2.5-Coder-3B, error bars span the two model-specific values.

models, and it best preserves the code surrounding the removed secrets (ForgetEval 0.996). When per-model tuning is affordable, the plain forget objectives offer stronger forget quality at roughly half the energy (PROD on Llama 3.2 3B, NPO on Qwen 2.5 Coder) but neither transfers its top rank to the other model. Under tight resource budgets, plain GA at an early epoch is about four times cheaper and remains competitive, but demands precise early stopping. Retain objectives on GA should be avoided, as they accelerate rather than prevent utility collapse. Finally, regardless of technique, failures concentrate on code-located secrets, making them the primary residual risk to check after unlearning.

Effect of Parameter Update Scope

All preceding unlearning experiments update only LoRA adapter parameters. We therefore conduct a focused ablation to examine whether expanding the parameter-update scope changes the unlearning–utility trade-off. Specifically, we compare LoRA-based and full-parameter GA on Qwen2.5-Coder-3B using the same dataset. Because LoRA and full-parameter optimization may require different learning rates [64, 65], we tune the full-parameter configuration separately. The search covers learning rates from 5×10^{-7} to 5×10^{-5} ; selected trajectories are shown in Appendix 1. Based on this search, we use 2×10^{-5} for the full-parameter comparison. Table 5.3 reports changes relative to LoRA-based GA at matched epochs. Full-parameter GA provides a substantial early advantage, improving forget quality by 7.62 pp and UtilityEval by 1.01 pp at the first epoch. This advantage largely disappears at the second epoch and reverses at the third, where forget quality and UtilityEval decrease by 2.78 and 2.02 pp, respectively. This pattern suggests that updating all parameters speeds up early forgetting but reduces the range over which a favorable unlearning–utility trade-off is maintained.

In contrast, full-parameter GA consistently improves HumanEval, with its advantage increasing from 1.22 pp during the first two epochs to 4.88 pp at the third epoch. This divergence

Table 5.2: ForgetEval pass@10 scores at the best epoch of each technique (in parentheses), chosen per model by the equal-weight mean of Forget Quality pass@10 ($1 - \text{chrF}$) and UtilityEval pass@10, averaged over Llama-3.2-3B and Qwen2.5-Coder-3B. The error denotes the deviation of each model from the mean.

Technique	Llama-3.2-3B	Qwen2.5-Coder-3B	Mean $\pm \Delta$
NPO+KL	1.000 (e2)	0.992 (e1)	0.996 ± 0.004
NPO+GD	0.976 (e2)	1.000 (e3)	0.988 ± 0.012
GA	0.952 (e2)	0.952 (e3)	0.952 ± 0.000
PROD+KL	0.984 (e1)	0.904 (e2)	0.944 ± 0.040
PROD	0.984 (e1)	0.896 (e2)	0.940 ± 0.044
NPO	0.976 (e1)	0.872 (e2)	0.924 ± 0.052
PROD+GD	0.872 (e1)	0.976 (e1)	0.924 ± 0.052
GA+KL	0.808 (e1)	0.968 (e1)	0.888 ± 0.080
GA+GD	0.760 (e1)	0.984 (e1)	0.872 ± 0.112

Table 5.3: Full-parameter GA relative to LoRA-based GA on Qwen2.5-Coder-3B.

Epoch	Δ Forget Quality (pp)	Δ UtilityEval (pp)	Δ HumanEval (pp)	Δ GPU Energy (%)	Δ Time (%)
1	+7.62	+1.01	+1.22	–	–
2	+0.13	0.00	+1.22	–	–
3	–2.78	–2.02	+4.88	–	–
<i>Mean / full run</i>	+1.66	–0.34	+2.44	+39.0	+69.5

All changes are calculated as full-parameter GA minus LoRA-based GA. All evaluation metrics use pass@10. Forget Quality is calculated as $1 - \text{chrF}$. The final row reports the arithmetic mean of the performance differences across the three matched epochs. GPU-energy and time differences cover the complete three-epoch runs.

is particularly notable because HumanEval measures general code-generation ability, whereas UtilityEval captures utility acquired during fine-tuning. At the third epoch, full-parameter GA improves HumanEval by 4.88 pp while reducing UtilityEval by 2.02 pp relative to LoRA. This pattern may indicate that full-parameter unlearning better preserves general model capabilities while progressively degrading later acquired utility. Full-parameter unlearning nevertheless requires substantially greater computational resources consistent with the established efficiency advantages of LoRA [66–68]. It increases measured GPU energy by 39.0% and training duration by 69.5%. This makes full-parameter GA less attractive for repeated or continual unlearning.

5.2 RQ2: Transferability of Machine Unlearning to Code Units

This research question examines whether the unlearning effectiveness and utility retention observed for secret unlearning transfer to structured code units. Instead of isolating and unlearning secrets within functions and classes, we unlearn entire code segments, simulating the removal of copyrighted code. As Table 5.4 shows, the results differ fundamentally once whole code units are subjected to unlearning, calling the practical usability of these techniques into question. For

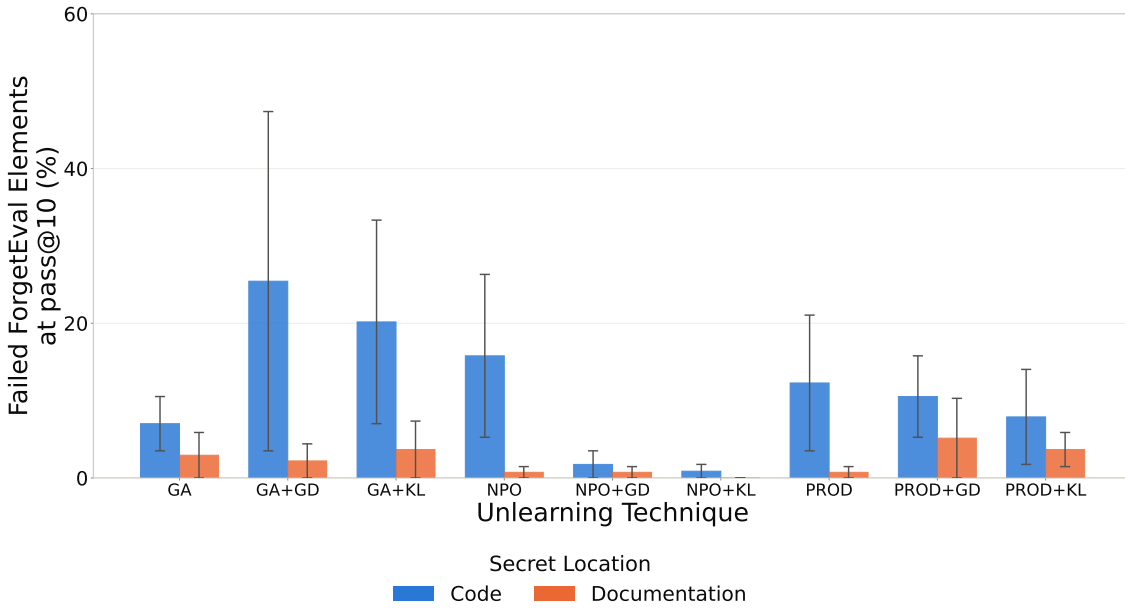


Figure 5.3: ForgetEval pass@10 failure rates by secret location for the selected secret-setting checkpoints. Bars report the mean failure rate across Llama-3.2-3B and Qwen2.5-Coder-3B, while error bars span the two model-specific rates. Results use the 125 tasks retained after baseline filtering, comprising 57 secrets located in code and 68 in documentation. For each model–technique pair, the checkpoint maximizing the equally weighted mean of UtilityEval and forget quality was selected.

GA, the familiar trend of stronger forgetting at the cost of utility persists across epochs, but the utility degradation is far more severe: by epoch 3, the GD and KL variants retain at most 7.1% utility, making the models effectively unusable. Base GA preserves utility best on both models, retaining at least 64.6% at epoch 3, yet its forget quality peaks at only 53.6%, far below the 96.9% in the secret-unlearning setup. Similar to secret unlearning, NPO yields the most stable behavior: its retain-objective variants hold utility above 92.9% at every epoch, and even base NPO never falls below 62.6%. This stability, however, comes at the cost of minimal forgetting, with forget quality never exceeding 23.4% across the NPO family. PROD exhibits the most severe degradation, reducing utility to exactly zero at every epoch on both models. We attribute this to its probability-distribution-based objective penalizing necessary syntactic structure, which results in unexecutable code, such as the generation in Listing 5.1. The gap between code-unit and secret unlearning is further emphasized in Figure 5.4. Secret-unlearning results cluster in the top-right corner, indicating a favorable trade-off between forget quality and utility retention, whereas code-unit configurations that reach comparable forget quality lose almost all utility, and those that match the secret setup’s utility achieve substantially lower forget quality. To ensure that the observed gap is not driven by differences in evaluation metrics, we recompute all similarities with normalized Levenshtein distance instead of chrF and BLEU and observe the same trends (Appendix Figure 2).

```

1  def synthetic_eval_function_aiF2M9(
2      on_time_delivery_pct: float,
3      quality_ppm: float,
4      cost_variance_pct: float,
5      responsiveness_hours: float,
6      open_audit_findings: int,
7      sustainability_score: float,
8      capacity_utilization_pct: float,
9      financial_risk_label: str,
10     strategic_partner: bool,
11     alternate_source_available: bool,
12     corrective_action_open: bool,
13     warranty_claim_rate: float,
14     innovation_points: int,
15     country_risk_level: str,
16     days_since_last_review: int,
17     probation_threshold_grade: str
18 ) -> str:
19     """
20     Assign a supplier scorecard grade.
21     Grades are A, B, C, probation, or exit_review. Delivery, quality, cost,
22     responsiveness, audits, sustainability, capacity, financial risk,
23     warranty, innovation, country risk, and review recency influence the
24     outcome.
25     Strategic partners receive a small lift, but open corrective action or
26     severe
27     financial risk can force probation or exit review.
28     """
29     <|file_sep|><|fim_prefix|>/src/retain_function_GGz1x5.py<|fim_suffix|>
30     lif password_length >= len(set(characters)):
31         password_strength = "strong"
32     <|file_sep|>/src/random_integers.py
33     """Compose simple graphs from nodes and edges."""

```

Listing 5.1: PROD-unlearned generation from Qwen2.5-Coder-3B at epoch 1 (UtilityEval, code-unit setting; highlighted segment). The model generates a file-separator token, hallucinates an unrelated file path, resulting in an unexecutable solution.

Takeaway: None of the evaluated techniques is usable for unlearning whole code units. PROD destroys utility entirely, GA collapses once forgetting is effective, and NPO preserves utility only by achieving negligible forget quality. The favorable trade-offs of the secret setup do not transfer from localized segments to entire code units. Reliable unlearning of copyrighted code therefore remains an open challenge, and existing techniques are not yet suitable for this purpose.

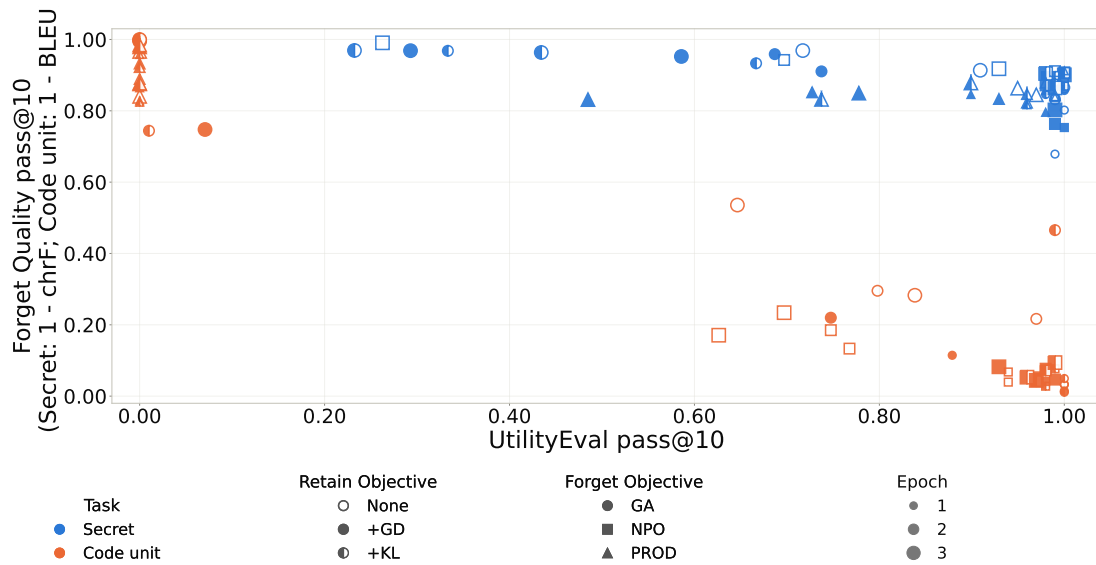


Figure 5.4: Pass@10 utility-forgetting trade-offs for all secret and code-unit checkpoints across both models. Colors indicate the task, shapes the forget objective, fills the retain objective, and marker sizes the epoch. forget quality is $1 - \text{chrF}$ for secret and $1 - \text{BLEU}$ for code unit; higher values on both axes are preferred.

Table 5.4: Code-unit unlearning results at every epoch. Each result cell reports Forget Quality / UtilityEval, where forget quality uses $1 - \text{BLEU}$.

Model	Technique	pass@1			pass@5			pass@10		
		Epoch 1	Epoch 2	Epoch 3	Epoch 1	Epoch 2	Epoch 3	Epoch 1	Epoch 2	Epoch 3
Llama-3.2-3B	GA	0.094 / 0.973	0.388 / 0.808	0.447 / 0.674	0.049 / 1.000	0.244 / 0.935	0.315 / 0.816	0.034 / 1.000	0.217 / 0.970	0.283 / 0.838
	GA+GD	0.191 / 0.796	0.998 / 0.000	0.998 / 0.000	0.128 / 0.875	0.998 / 0.000	0.998 / 0.000	0.115 / 0.879	0.998 / 0.000	0.998 / 0.000
	GA+KL	0.035 / 0.969	0.914 / 0.005	0.995 / 0.000	0.013 / 1.000	0.801 / 0.010	0.995 / 0.000	0.011 / 1.000	0.744 / 0.010	0.995 / 0.000
	NPO	0.175 / 0.813	0.388 / 0.512	0.454 / 0.455	0.090 / 0.918	0.221 / 0.707	0.265 / 0.639	0.068 / 0.939	0.185 / 0.747	0.234 / 0.697
	NPO+GD	0.138 / 0.856	0.123 / 0.825	0.197 / 0.721	0.079 / 0.956	0.085 / 0.963	0.105 / 0.899	0.057 / 0.970	0.077 / 0.980	0.082 / 0.929
	NPO+KL	0.135 / 0.824	0.137 / 0.744	0.172 / 0.723	0.039 / 0.969	0.062 / 0.946	0.078 / 0.925	0.028 / 0.980	0.042 / 0.970	0.053 / 0.960
	PROD	0.941 / 0.000	0.959 / 0.000	0.920 / 0.000	0.889 / 0.000	0.906 / 0.000	0.862 / 0.000	0.864 / 0.000	0.889 / 0.000	0.840 / 0.000
	PROD+GD	0.922 / 0.000	0.981 / 0.000	0.943 / 0.000	0.850 / 0.000	0.949 / 0.000	0.894 / 0.000	0.828 / 0.000	0.932 / 0.000	0.875 / 0.000
	PROD+KL	0.917 / 0.000	0.977 / 0.000	0.943 / 0.000	0.849 / 0.000	0.938 / 0.000	0.891 / 0.000	0.824 / 0.000	0.923 / 0.000	0.877 / 0.000
Qwen2.5-Coder-3B	GA	0.096 / 0.974	0.496 / 0.679	0.607 / 0.474	0.035 / 0.993	0.355 / 0.774	0.546 / 0.599	0.032 / 1.000	0.295 / 0.798	0.536 / 0.646
	GA+GD	0.039 / 0.982	0.354 / 0.531	0.831 / 0.044	0.016 / 1.000	0.253 / 0.697	0.772 / 0.063	0.015 / 1.000	0.220 / 0.747	0.748 / 0.071
	GA+KL	0.168 / 0.988	0.545 / 0.968	1.000 / 0.000	0.072 / 1.000	0.481 / 0.989	1.000 / 0.000	0.049 / 1.000	0.465 / 0.990	1.000 / 0.000
	NPO	0.103 / 0.878	0.265 / 0.527	0.305 / 0.390	0.045 / 0.939	0.162 / 0.728	0.190 / 0.576	0.039 / 0.939	0.133 / 0.768	0.171 / 0.626
	NPO+GD	0.112 / 0.928	0.176 / 0.940	0.157 / 0.912	0.051 / 0.980	0.053 / 0.989	0.063 / 0.969	0.041 / 0.980	0.046 / 0.990	0.044 / 0.970
	NPO+KL	0.233 / 0.938	0.230 / 0.891	0.293 / 0.870	0.107 / 0.988	0.100 / 0.974	0.133 / 0.974	0.077 / 0.990	0.072 / 0.980	0.094 / 0.990
	PROD	0.996 / 0.000	0.996 / 0.000	0.993 / 0.000	0.988 / 0.000	0.987 / 0.000	0.977 / 0.000	0.981 / 0.000	0.979 / 0.000	0.966 / 0.000
	PROD+GD	0.995 / 0.000	0.997 / 0.000	0.995 / 0.000	0.986 / 0.000	0.988 / 0.000	0.986 / 0.000	0.980 / 0.000	0.983 / 0.000	0.979 / 0.000
	PROD+KL	0.992 / 0.000	0.995 / 0.000	0.998 / 0.000	0.970 / 0.000	0.986 / 0.000	0.991 / 0.000	0.958 / 0.000	0.978 / 0.000	0.985 / 0.000

5.3 RQ3: Effects of Retain-Set Size and Forget–Retain Objective Ordering

After observing the practical applicability of unlearning secrets injected into code segments, this research question more extensively investigates whether the trade-off among forget quality, utility retention, and energy consumption can be further optimized by changing the retain-set size and ordering the forget and retain objectives. The results of these experiments can be seen in Figure 5.5. Here, we observe that the main driving factor behind the results is the forget-objective family.

GA-based techniques. For GA, increasing the retain-set size and ordering the objectives increase utility stability, especially at later epochs. Mean UtilityEval increases under every tested GA configuration, ranging from +4.0 percentage points (pp) for double retain with GD to +32.8 pp for retain first with KL. Forget first and retain first provide similar utility-retention benefits, however forget first sacrifices substantially less forget quality, making it a better fit for the overall trade-off. With GD, forget first improves utility by 22.1 pp while reducing forget quality by 4.3 pp, whereas retain first improves utility by 23.6 pp but reduces forget quality by 12.3 pp. Similarly, with KL, forget first improves utility by 27.9 pp with a 1.7 pp forget quality reduction, while retain first improves utility by 32.8 pp but reduces forget quality by 7.9 pp. Double retain does not increase utility as much on average, but it provides a slight forget quality increase of 0.9 pp with GD and 2.1 pp with KL. All of these configurations also increase mean retain quality relative to random order ranging from 1.0 to 5.6 pp. Moreover, the KL retain objective generally provides a better trade-off by preserving more utility and retain quality while producing a smaller reduction in forget quality than GD alternatives, particularly under forget-first ordering.

NPO-based techniques. Most NPO performance changes remain close to zero. Forget-first variants tend to provide slightly better forget quality with insignificant changes to model utility achieved by random ordering. In particular, forget first with KL changes UtilityEval, forget quality, and retain quality by +0.7, +1.2, and +0.3 pp, respectively. Forget first with GD improves forget quality by 1.8 pp while changing UtilityEval by only -0.2 pp. Neither retain first nor double retain provides a consistent additional benefit for NPO.

PROD-based techniques. PROD is highly sensitive to retain-set changes and is generally harmed by them. In contrast to GA, utility retention degrades when objectives are ordered or the retain-set size is increased, usually with either minimal or no improvement in forget quality. Retain first with GD produces the most severe degradation, reducing UtilityEval by 60.3 pp and retain quality by 26.0 pp. Forget first with KL provides the best trade-off among the evaluated PROD alternatives: it improves forget quality by 4.8 pp and retain quality by 0.8 pp, but UtilityEval still decreases by 6.2 pp. On top of that, retain quality also decreases for most PROD configurations.

Resource consumption. Interestingly, across both models, we observe that forget-first variants consume less GPU energy than retain-first variants. Forget first consumes approximately 8.0% less measured GPU energy with GD and 9.0% less with KL than random order. Retain first, in comparison, consumes 1.2% and 4.7% more GPU energy. Considering that forget first usually provides the better trade-off for all forget objective families, it represents a favorable choice, especially when combined with KL. Doubling the retain-set size also increases measured GPU-energy consumption by 76.3% with GD and 93.5% with KL, approaching twice the consumption of random order. Considering that increasing the retain-set size does not consistently improve the

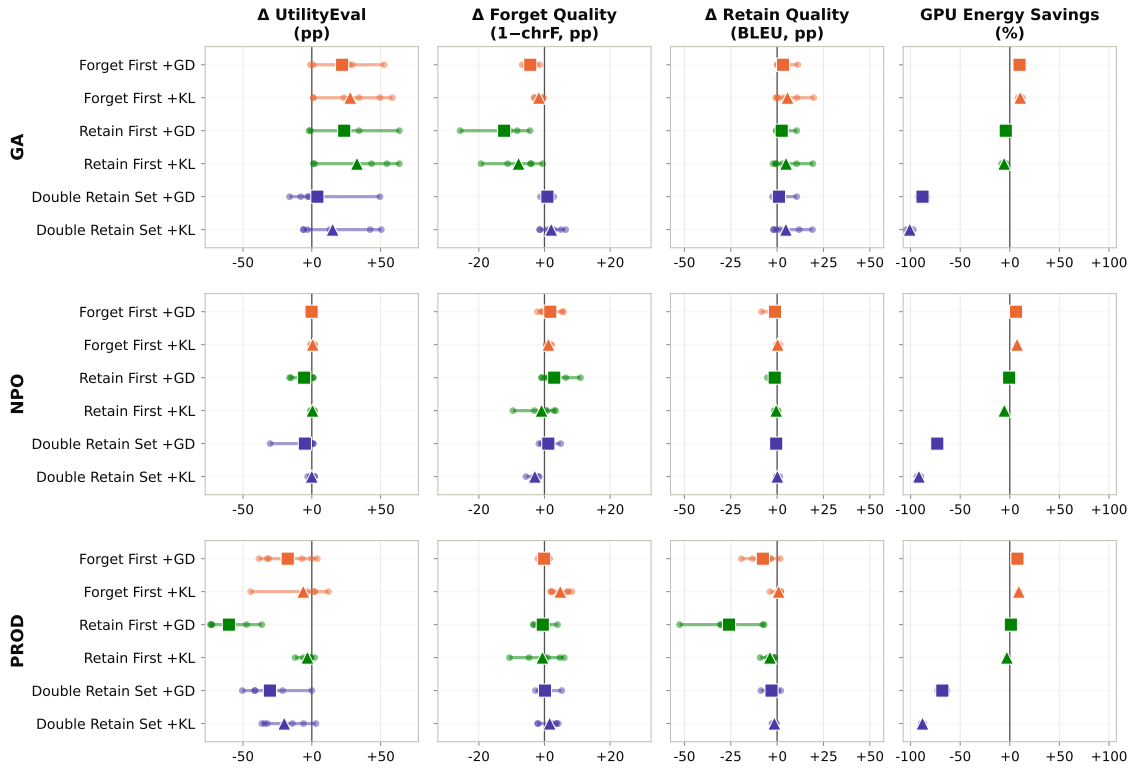


Figure 5.5: Effects of forget-retain objective ordering and double retain set size relative to random objective ordering for the secret setting. Performance differences are calculated against the corresponding random-order checkpoint for the same model, objective family, and epoch. Small points represent the six matched observations (two models across three epochs), large markers indicate their means, and horizontal lines show the observed ranges.

trade-off, this additional computational cost makes the double-retain setting less practical. The energy comparisons should nevertheless be interpreted as observational because the random-order and alternative runs were measured using different A100 variants.

Takeaway: Objective ordering has a more consistent influence on the unlearning trade-off than increasing the retain-set size, although its effect remains strongly dependent on the forget-objective family. Forget-first ordering with KL provides the most favorable overall balance between forgetting, utility preservation, retain quality, and measured GPU-energy consumption. In contrast, doubling the retain set nearly doubles energy use without consistently improving performance. GA benefits most from objective ordering, NPO remains comparatively stable, and PROD is frequently harmed, demonstrating that these procedural choices must be selected jointly with the unlearning objective.

Limitations and Threats to Validity

Internal validity. A first threat to internal validity concerns the dataset curation stage, particularly the similarity between code units and the extent to which the resulting code represents realistic cases while minimizing overlap with code knowledge already present in the models. To mitigate this threat, we apply threshold-based filtering across multiple dimensions and conduct an initial memorization analysis of the curated samples. A second threat concerns the experimental configuration. Throughout the study, we use hyperparameters that have been validated in prior work; however, these values may not be equally suitable for our forget and retain sets. We try to reduce this risk by doing small scale grid searches. Fine-tuning and unlearning may also introduce variance. Since we do not run multiple seeds, observed differences between methods or ordering strategies may partly reflect randomness.

The evaluation setup introduces additional threats. Although the synthetically generated test suite increases diversity by covering different inputs, and edge cases, this diversity may not accurately represent the distribution of real-world programs and developer-written tests. We try to mitigate this by adding additional mutation hardening step by utilizing mutation scores as a guide. Moreover, CodeCarbon follows the general pattern of energy use, but consistently underestimates consumption by about 20–30% which suggests that CodeCarbon is useful for relative comparisons, but less reliable for exact energy consumption [69]. This is the main reason why we interpret energy results as general trends rather than definitive truth. Finally, one of the most important internal threats is the choice of metrics. Since there is no established metric on unlearning field we decided to use text similarity metrics BLEU and chrF which may not effectively quantify the unlearning effect.

External validity. The most significant external threat to validity is the use of a synthetic dataset rather than code units collected from open-source repositories. However, the generated code is expected to exhibit statistical characteristics similar to human-written code because the LLMs used for generation were trained on large-scale corpora of publicly available source code. Therefore, the synthetic code may follow a distribution comparable to real-world code, although this similarity cannot be guaranteed. Another limitation is the selection of models. Due to resource limitations, we evaluate only two small LLMs with 3B parameters. To address this generalizability concern, we evaluate one general-purpose LLM and one code-specific LLM. Despite this, to assess generalization more comprehensively, future evaluations should include additional LLMs and CodeLLMs from different model families and across a wider range of model sizes. In addition, this study only considers Python, and its generalizability to other programming languages is therefore uncertain. Since we focus only on functions and classes, it is also unclear whether the findings hold for arbitrary code fragments without a clear structure.

Conclusion and Future Work

The main goal of this thesis is to explore the forget quality, utility retention, and energy consumption trade-offs of applying MU techniques to remove undesired data from code, ranging from secrets and PII embedded in code to whole semantically and syntactically structured code units. To analyze this, we conducted two separate unlearning experiments on a synthetically generated dataset, targeting secrets injected into functions and classes, and comparing this against unlearning the corresponding code units as a whole. Our results indicate a significant gap between the secret and code-unit settings. Existing MU methods work as intended for removing secrets: NPO variants, particularly NPO+KL, provide the most stable trade-off in most scenarios, balancing forget quality and utility retention while increasing resource consumption only slightly compared to other methods. Forget-only variants also offer a favorable trade-off under tighter resource constraints; however, their behavior is less consistent across models and can change considerably over epochs, especially for GA. In contrast, code-unit unlearning configurations fail to achieve comparable success. When forget quality equivalent to the secret setting is reached, utility degrades to impractical or unusable levels, and preserving utility largely makes the unlearning process ineffective.

The second objective of this thesis is to analyze how retain-set size and the ordering of forget and retain objectives interact with the unlearning trade-off, a novel and largely unexplored dimension in the MU literature. Our results suggest that the forget objective remains the dominant factor determining the outcome. GA-based approaches become more stable with a larger retain set and ordered objectives, though this generally comes at the cost of some forget quality. For NPO, these changes do not significantly affect the outcome. Forget-first ordering combined with KL stands out as particularly competitive across both forget families, whereas PROD responds negatively to these modifications and typically loses both forget quality and utility retention.

Overall, the results demonstrate that existing MU techniques do not transfer to whole code units. This is concerning, since copyrighted-code removal is one of the more attractive applications of these techniques for code generation tasks, and the same capability could extend to removing harmful code segments, such as malware or code enabling cybersecurity attacks. This suggests that future work should focus on new algorithms capable of achieving this without compromising the coding capabilities of the model. Beyond this, existing MU techniques mostly target the unlearning of specific, isolated elements; it would be valuable to extend this to unlearning whole concepts rather than singular instances. In this work, we aimed for an uncontaminated and controlled experimental design through synthetically generated code; future work could extend this design to additional scenarios, such as unlearning deprecated code or entire repositories.

Bibliography

- [1] D. Zhang, P. Finckenberg-Broman, T. Hoang, S. Pan, Z. Xing, M. Staples, and X. Xu, "Right to be forgotten in the era of large language models: Implications, challenges, and solutions," 2024.
- [2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, and N. J. et al., "Evaluating large language models trained on code," 2021.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020.
- [4] C. Zhang, H. Zhong, K. Zhang, C. Chai, R. Wang, X. Zhuang, T. Bai, Q. Jiantao, L. Cao, J. Fan, Y. Yuan, G. Wang, and C. He, "Harnessing diversity for important data selection in pretraining large language models," in *The Thirteenth International Conference on Learning Representations*, 2025.
- [5] C. Zhang, C. Yang, Q. Tan, J. Liu, A. Li, Y. Wang, J. Lu, J. Wang, and G. Yuan, "Towards memory-efficient and sustainable machine unlearning on edge using zeroth-order optimizer," in *Proceedings of the Great Lakes Symposium on VLSI 2025, GLSVLSI '25*, (New York, NY, USA), p. 227–232, Association for Computing Machinery, 2025.
- [6] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, T. Liu, M. Tian, D. Kocetkov, A. Zucker, Y. Belkada, Z. Wang, Q. Liu, D. Abulkhanov, I. Paul, Z. Li, W.-D. Li, M. Risdal, J. Li, J. Zhu, T. Y. Zhuo, E. Zheltonozhskii, N. O. O. Dade, W. Yu, L. Krauß, N. Jain, Y. Su, X. He, M. Dey, E. Abati, Y. Chai, N. Muennighoff, X. Tang, M. Oblokulov, C. Akiki, M. Marone, C. Mou, M. Mishra, A. Gu, B. Hui, T. Dao, A. Zebaze, O. Dehaene, N. Patry, C. Xu, J. McAuley, H. Hu, T. Scholak, S. Paquet, J. Robinson, C. J. Anderson, N. Chapados, M. Patwary, N. Tajbakhsh, Y. Jernite, C. M. Ferrandis, L. Zhang, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries, "StarCoder 2 and the stack v2: The next generation," 2024.
- [7] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," 2024.
- [8] A. Al-Kaswan, M. Izadi, and A. van Deursen, "Traces of Memorisation in Large Language Models for Code," in *Proceedings of the IEEE/ACM 46th International Conference on Software*

- Engineering*, ICSE '24, (New York, NY, USA), Association for Computing Machinery, 2024. event-place: Lisbon, Portugal.
- [9] J. He, M. Vero, G. Krasnopolska, and M. Vechev, "Instruction tuning for secure code generation," in *Proceedings of the 41st International Conference on Machine Learning, ICML'24*, (Vienna, Austria), JMLR.org, 2024.
 - [10] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," 2020.
 - [11] S. Liu, Y. Yao, J. Jia, S. Casper, N. Baracaldo, P. Hase, Y. Yao, C. Y. Liu, X. Xu, H. Li, K. R. Varshney, M. Bansal, S. Koyejo, and Y. Liu, "Rethinking machine unlearning for large language models," 2024.
 - [12] A. Ginart, M. Y. Guan, G. Valiant, and J. Zou, "Making ai forget you: Data deletion in machine learning," 2019.
 - [13] S. Neel, A. Roth, and S. Sharifi-Malvajerdi, "Descent-to-delete: Gradient-based methods for machine unlearning," in *Proceedings of the 32nd International Conference on Algorithmic Learning Theory*, vol. 132 of *Proceedings of Machine Learning Research*, pp. 931–962, PMLR, 2021.
 - [14] European Parliament and Council of the European Union, "Regulation (eu) 2016/679 of the european parliament and of the council." Official Journal of the European Union, L 119, 2016. General Data Protection Regulation, Articles 12(3) and 17.
 - [15] European Union, *Charter of Fundamental Rights of the European Union*, vol. 53. Brussels: European Union, 2010.
 - [16] GDPR.eu, "Right to be forgotten." . Accessed March 2, 2026.
 - [17] M. V. Waerebeke, M. Lorenzi, G. Neglia, and K. Scaman, "When to forget? complexity trade-offs in machine unlearning," 2025.
 - [18] R. Geng, M. Geng, S. Wang, H. Wang, Z. Lin, and D. Dong, "Mitigating sensitive information leakage in llms4code through machine unlearning," 2025.
 - [19] X. Jiang, Y. Dong, Z. Fang, Y. Ma, T. Wang, R. Cao, B. Li, Z. Jin, W. Jiao, Y. Li, and G. Li, "Large language model unlearning for source code," 2025.
 - [20] Z. Chu, Y. Wan, Z. Zhang, D. Wang, Z. Yang, H. Zhang, P. Zhou, X. Shi, H. Jin, and D. Lo, "Scrub it out! erasing sensitive memorization in code language models via machine unlearning," *arXiv preprint arXiv:2509.13755*, 2025.
 - [21] N. Li, C. Zhou, Y. Gao, H. Chen, A. Fu, Z. Zhang, and Y. Shui, "Machine unlearning: Taxonomy, metrics, applications, challenges, and prospects," 2024.
 - [22] Z. Liu, G. Dou, Z. Tan, Y. Tian, and M. Jiang, "Machine unlearning in generative ai: A survey," 2024.
 - [23] A. Warnecke, L. Pirch, C. Wressnegger, and K. Rieck, "Machine unlearning of features and labels," 2023.
 - [24] J. Yao, E. Chien, M. Du, X. Niu, T. Wang, Z. Cheng, and X. Yue, "Machine unlearning of pre-trained large language models," 2024.

- [25] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* (P. Isabelle, E. Charniak, and D. Lin, eds.), (Philadelphia, Pennsylvania, USA), pp. 311–318, Association for Computational Linguistics, July 2002.
- [26] M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," in *Proceedings of the Tenth Workshop on Statistical Machine Translation* (O. Bojar, R. Chatterjee, C. Federmann, B. Haddow, C. Hokamp, M. Huck, V. Logacheva, and P. Pecina, eds.), (Lisbon, Portugal), pp. 392–395, Association for Computational Linguistics, Sept. 2015.
- [27] Y. Liang, Q. Li, S. Wang, and W. Cao, "When forgetting builds reliability: Llm unlearning for reliable hardware code generation," 2025.
- [28] W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer, "Detecting pretraining data from large language models," 2024.
- [29] K. Tirumala, A. H. Markosyan, L. Zettlemoyer, and A. Aghajanyan, "Memorization without overfitting: Analyzing the training dynamics of large language models," 2022.
- [30] J. Jang, D. Yoon, S. Yang, S. Cha, M. Lee, L. Logeswaran, and M. Seo, "Knowledge unlearning for mitigating privacy risks in language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (A. Rogers, J. Boyd-Graber, and N. Okazaki, eds.), (Toronto, Canada), pp. 14389–14408, Association for Computational Linguistics, July 2023.
- [31] P. Maini, Z. Feng, A. Schwarzschild, Z. C. Lipton, and J. Z. Kolter, "Tofu: A task of fictitious unlearning for llms," 2024.
- [32] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, and S. A. et al., "Gpt-4 technical report," 2024.
- [33] H. Husain, H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *ArXiv*, vol. abs/1909.09436, 2019.
- [34] M. Rahman, S. Khatoonabadi, and E. Shihab, "Openclassgen: A large-scale corpus of real-world python classes for llm research," 2025.
- [35] A. LeClair and C. McMillan, "Recommendations for datasets for source code summarization," *ArXiv*, vol. abs/1904.02660, 2019.
- [36] V. Majdinasab, A. Nikanjam, and F. Khomh, "Trained without my consent: Detecting code inclusion in language models trained on code," *ACM Transactions on Software Engineering and Methodology*, vol. 34, p. 1–46, Apr. 2025.
- [37] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, and A. A. et al., "Openai gpt-5 system card," 2026.
- [38] J. McCabe, Thomas J., "Software quality metrics to identify risk." Presentation to the Department of Homeland Security Software Assurance Working Group, Jan. 2008. Last edited for content in November 2008. Archived from the original on March 29, 2022.
- [39] T. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [40] G. Petrović, M. Ivanković, G. Fraser, and R. Just, "Practical mutation testing at scale: A view from google," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3900–3912, 2022.

- [41] G. Grano, F. Palomba, and H. C. Gall, "Lightweight assessment of test-case effectiveness using source-code-quality indicators," *IEEE Transactions on Software Engineering*, vol. 47, no. 4, pp. 758–774, 2021.
- [42] M. Gligoric, V. Jagannath, and D. Marinov, "Mutmut: Efficient exploration for mutation testing of multithreaded code," in *2010 Third International Conference on Software Testing, Verification and Validation*, pp. 55–64, 2010.
- [43] B. H. Smith and L. Williams, "On guiding the augmentation of an automated test suite via mutation analysis," *Empirical Softw. Engg.*, vol. 14, p. 341–369, June 2009.
- [44] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "Codebleu: a method for automatic evaluation of code synthesis," 2020.
- [45] N. Godey, É. Clergerie, and B. Sagot, "Anisotropy is inherent to self-attention in transformers," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)* (Y. Graham and M. Purver, eds.), (St. Julian's, Malta), pp. 35–48, Association for Computational Linguistics, Mar. 2024.
- [46] J. Mu, S. Bhat, and P. Viswanath, "All-but-the-top: Simple and effective postprocessing for word representations," 2018.
- [47] L. Ai, Z. Gong, H. Deshpande, A. Johnson, E. Phung, A. Emami, and J. Hirschberg, "Novascore: A new automated metric for evaluating document level novelty," 2024.
- [48] A. Spiesberger, J. J. Vazquez, N. Pochinkov, T. Gavenčiak, P. Grietzer, G. Leech, and N. Schoots, "Soft contamination means benchmarks test shallow generalization," 2026.
- [49] M. Mishra, M. Stallone, G. Zhang, Y. Shen, A. Prasad, A. M. Soria, M. Merler, P. Selvam, S. Surendran, S. Singh, M. Sethi, X.-H. Dang, P. Li, K.-L. Wu, S. Zawad, A. Coleman, M. White, M. Lewis, R. Pavuluri, Y. Koyfman, B. Lublinsky, M. de Bayser, I. Abdelaziz, K. Basu, M. Agarwal, Y. Zhou, C. Johnson, A. Goyal, H. Patel, Y. Shah, P. Zerfos, H. Ludwig, A. Munawar, M. Crouse, P. Kapanipathi, S. Salaria, B. Calio, S. Wen, S. Seelam, B. Belgodere, C. Fonseca, A. Singhee, N. Desai, D. D. Cox, R. Puri, and R. Panda, "Granite code models: A family of open foundation models for code intelligence," 2024.
- [50] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "Sourcerercc: Scaling code clone detection to big-code," in *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*, pp. 1157–1168, 2016.
- [51] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, and A. V. et al., "The llama 3 herd of models," 2024.
- [52] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024.
- [53] Y. Yao, X. Xu, and YangLiu, "Large language model unlearning," in *Advances in Neural Information Processing Systems* (A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, eds.), vol. 37, pp. 105425–105475, Curran Associates, Inc., 2024.
- [54] R. Zhang, L. Lin, Y. Bai, and S. Mei, "Negative preference optimization: From catastrophic collapse to effective unlearning," 2024.
- [55] B. Liu, Q. Liu, and P. Stone, "Continual learning and private unlearning," 2022.

- [56] J. Chen and D. Yang, “Unlearn what you want to forget: Efficient unlearning for llms,” 2023.
- [57] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramèr, and C. Zhang, “Quantifying memorization across neural language models,” 2023.
- [58] D. Aspinall and M. Just, ““give me letters 2, 3 and 6!”: Partial password implementations and attacks,” in *Financial Cryptography and Data Security* (A.-R. Sadeghi, ed.), (Berlin, Heidelberg), pp. 126–143, Springer Berlin Heidelberg, 2013.
- [59] U.S. Copyright Office, “What does copyright protect?.”, n.d. Frequently Asked Questions. Accessed: 2026-06-23.
- [60] J. R. Autry, “Toward a definition of striking similarity in infringement actions for copyrighted musical works,” *Journal of Intellectual Property Law*, vol. 10, no. 1, p. 113, 2002.
- [61] M. Post, “A call for clarity in reporting BLEU scores,” in *Proceedings of the Third Conference on Machine Translation: Research Papers* (O. Bojar, R. Chatterjee, C. Federmann, M. Fishel, Y. Graham, B. Haddow, M. Huck, A. J. Yepes, P. Koehn, C. Monz, M. Negri, A. N  v  ol, M. Neves, M. Post, L. Specia, M. Turchi, and K. Verspoor, eds.), (Brussels, Belgium), pp. 186–191, Association for Computational Linguistics, Oct. 2018.
- [62] B. Courty, V. Schmidt, S. Luccioni, Goyal-Kamal, MarionCoutarel, B. Feld, J. Lecourt, Liam-Connell, A. Saboni, Inimaz, supatomic, M. L  val, L. Blanche, A. Cruveiller, ouminasara, F. Zhao, A. Joshi, A. Bogroff, H. de Lavoreille, N. Laskaris, E. Abati, D. Blank, Z. Wang, A. Catovic, M. Alencon, M. St  chly, C. Bauer, L. O. N. de Ara  jo, JPW, and MinervaBooks, “mlco2/codecarbon: v2.4.1,” May 2024.
- [63] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021.
- [64] R. Shuttleworth, J. Andreas, A. Torralba, and P. Sharma, “Lora vs full fine-tuning: An illusion of equivalence,” 2025.
- [65] N. Chen, S. Villar, and S. Hayou, “Learning rate scaling across lora ranks and transfer to full finetuning,” 2026.
- [66] A. J. Shreshthi, S. Span  , L. D. Nunzio, R. L. Cesa, C. V. M. Re, and G. C. Cardarilli, “Parameter-efficient fine-tuning of llms for low-power sentiment analysis applications,” *JOIV : International Journal on Informatics Visualization*, 2026.
- [67] X. Sun, Y. Ji, B. Ma, and X. Li, “A comparative study between full-parameter and lora-based fine-tuning on chinese instruction data for instruction following large language model,” *ArXiv*, vol. abs/2304.08109, 2023.
- [68] J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *ArXiv*, vol. abs/2106.09685, 2021.
- [69] R. Fischer, “Ground-truthing ai energy consumption: Validating codecarbon against external measurements,” 2025.

Appendix

Table 1: Sensitivity of the Secret pass@10 best-epoch selection to a common UtilityEval and Forget Quality threshold τ . Epoch agreement is calculated among configurations remaining eligible at τ , relative to the 0.80/0.80 selection. Frontier retention is relative to its four Pareto-optimal points.

τ	Qualifying checkpoints	Eligible configurations	Same selected epoch	Frontier retained
0.65	47/54	18/18	17/18	4/4
0.70	43/54	18/18	17/18	4/4
0.75	39/54	18/18	17/18	4/4
0.80	35/54	18/18	18/18	4/4
0.85	22/54	13/18	12/13	4/4
0.90	8/54	5/18	5/5	3/4

Table 2: One-factor-at-a-time sensitivity of the Secret pass@10 selection. One threshold is varied while the other is fixed at 0.80. Epoch agreement is relative to the selected epochs at 0.80/0.80 and is reported among eligible configurations. Frontier retention is relative to the four Pareto-optimal points at 0.80/0.80.

τ	Vary UtilityEval ($\tau_{\text{FQ}} = 0.80$)			Vary Forget Quality ($\tau_{\text{U}} = 0.80$)		
	Eligible	Epoch match	Frontier	Eligible	Epoch match	Frontier
0.65	18/18	18/18	4/4	18/18	17/18	4/4
0.70	18/18	18/18	4/4	18/18	17/18	4/4
0.75	18/18	18/18	4/4	18/18	17/18	4/4
0.80	18/18	18/18	4/4	18/18	18/18	4/4
0.85	18/18	18/18	4/4	13/18	12/13	4/4
0.90	17/18	17/17	4/4	5/18	5/5	3/4

Table 3: Sensitivity of the Secret pass@10 best-epoch selection to baseline-derived filtering. Epoch agreement is relative to the filtered selection. Frontier retention is relative to its four Pareto-optimal points.

Result set	Qualifying checkpoints	Eligible configurations	Same selected epoch	Frontier retained
Filtered	35/54	18/18	18/18	4/4
Unfiltered	36/54	18/18	15/18	3/4

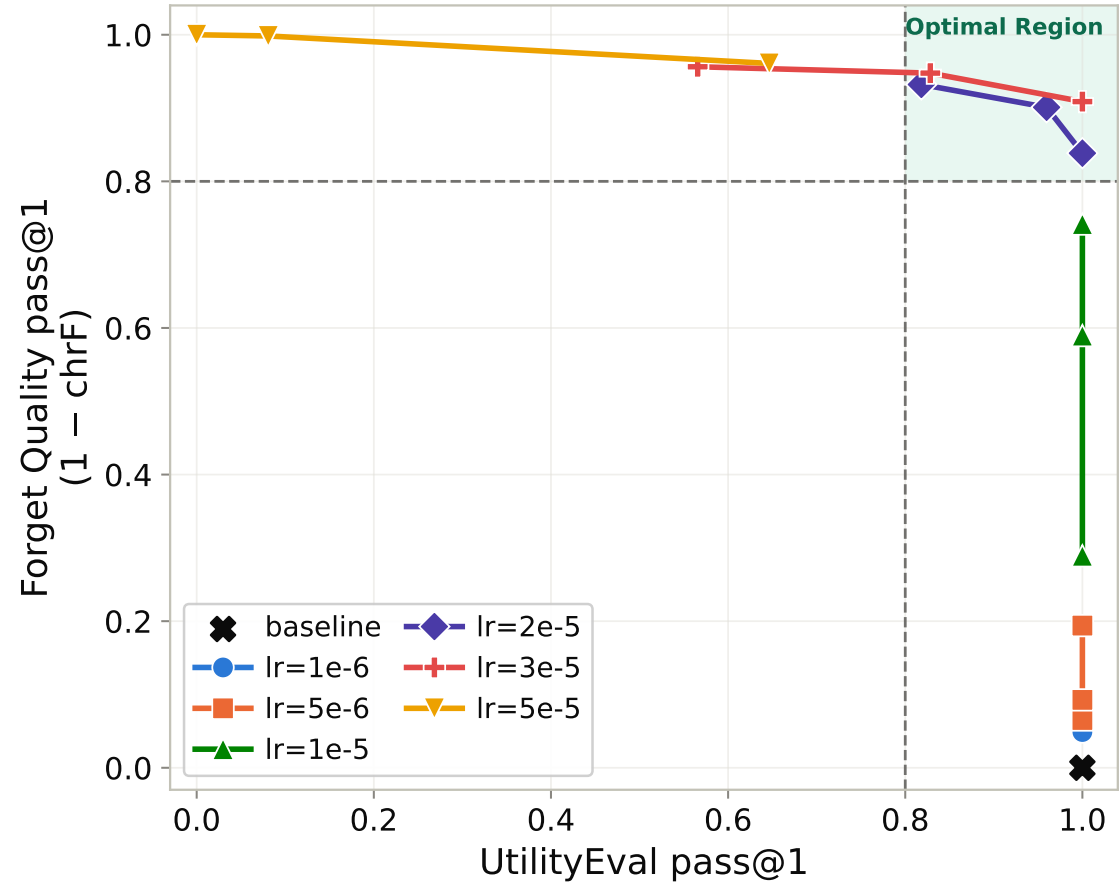


Figure 1: Learning-rate sensitivity of full-parameter GA on Qwen2.5-Coder-3B under the secret unlearning setting. Connected markers represent epochs 1–3.

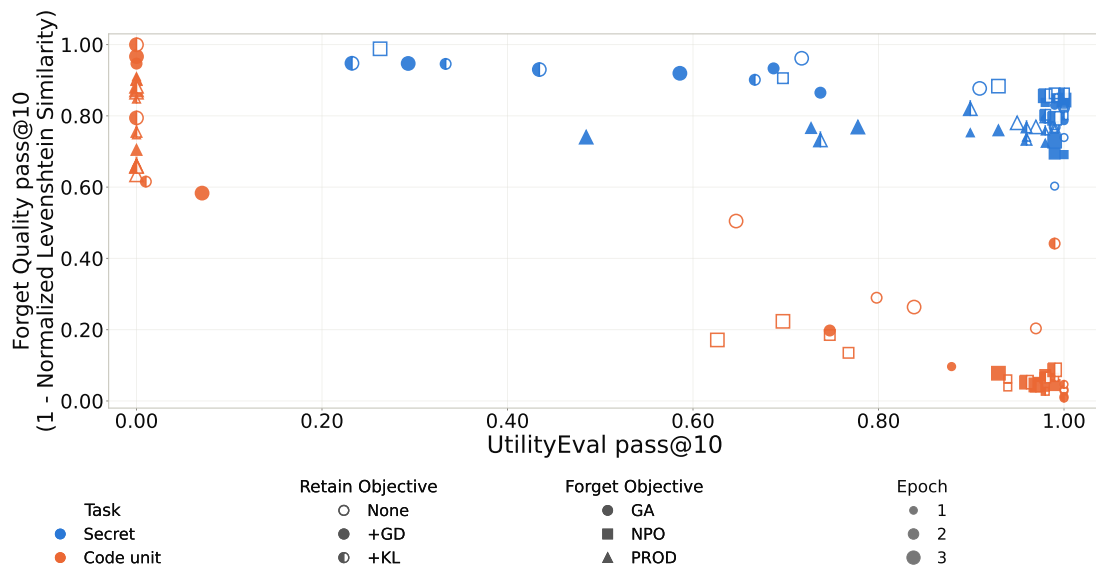


Figure 2: Pass@10 utility–forgetting trade-offs for all checkpoints across both models, using 1–normalized character-level Levenshtein similarity as a common forget quality measure.